Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет: инженерно-экономический
Кафедра: экономической информатики
Название дисциплины: объектно-ориентированное проектирование и программирование

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему

Разработка автоматизированной системы учета деятельности школы иностранных языков

Выполнила: студент группы 372302
Усова В.А.
Проверила: ассистент кафедры ЭИ
Примакович Л.В.

Минск 2024

СОДЕРЖАНИЕ

Введение 5
1 Обзор предметной области автоматизированной системы учета деятельности школы иностранных языков, программных аналогов, методов и алгоритмов решения поставленных задач 6
1.1 Описание деятельности школы иностранных языков 6
1.2 Обзор программных аналогов автоматизированной системы учета деятельности школы иностранных языков 10
1.3 Обзор методов и алгоритмов решения поставленных задач автоматизированной системы учета деятельности школы иностранных языков 12
2 Функциональное моделирование на основе стандарта IDEF0 14
3 Структура используемых данных 23
4 Описание созданных программных конструкций 25
5 Разработка и описание диаграммы классов 27
6 Разработка и описание диаграммы вариантов использования приложения 34
7 Блок-схема алгоритма работы всей программы и двух основных методов 38
7.1 Схема алгоритма работы всей программы 38
7.2 Схема алгоритма регистрации пользователя 41
7.3 Схема алгоритма авторизации пользователя 42
8 Описание алгоритма запуска приложения, его использования, результаты работы, тестирования обработки ошибок 44
8.1 Алгоритм запуска приложения 44
8.2 Руководство пользователя 44
8.3 Тестирование работы приложения 59
Заключение 62
Приложение А(обязательное) Листинг программы с комментариями 65
Приложение Б(обязательное) Диаграмма классов 78
Приложение В(обязательное) Диаграмма вариантов использования 82
Приложение Г(обязательное) Блок-схемы алгоритмов 83

ВВЕДЕНИЕ

В условиях динамично развивающегося общества, где знания иностранных языков становятся необходимым элементом для успешной коммуникации и профессиональной деятельности, школы иностранных языков занимают важную нишу в образовательной сфере. Эти учреждения обеспечивают студентов не только знаниями языков, но и навыками межкультурной коммуникации, что в свою очередь способствует их интеграции в глобальное сообщество. Однако с ростом числа студентов и увеличением объема операций, связанных с организацией учебного процесса, перед многими учебными заведениями возникает необходимость в автоматизации различных рабочих процессов.

На сегодняшний день многие школы иностранных языков сталкиваются с проблемой ручного учета, что приводит к значительным временным и ресурсным затратам, а также повышает вероятность возникновения ошибок.

Автоматизация этих процессов открывает новые возможности для оптимизации рабочего времени преподавателей и сотрудников учебных заведений, а также повышает качество взаимодействия между студентами и администрацией.

Целью данного курсового проекта является разработка автоматизированной системы учета деятельности школы иностранных языков, которая обеспечит интеграцию всех ключевых процессов учреждения в единую платформу. Это позволит не только облегчить управление учебным процессом, но и позволит улучшить качество взаимодействия с клиентами и оптимизировать ресурсы, что, в свою очередь, окажет положительное влияние на общую эффективность работы учебного заведения.

Для достижения поставленной цели в рамках работы необходимо решить несколько задач:

– провести анализ существующих информационных систем и их возможностей для учета образовательной деятельности;

– разработать проект архитектуры автоматизированной системы;

– выбрать подходящие технологии для ее реализации;

– предложить методы внедрения и оценки эффективности системы.

Одной из ключевых задач будет создание удобного и интуитивно понятного интерфейса для пользователей системы, что обеспечит ее успешное внедрение и дальнейшее использование.

В результате выполнения данного проекта будет предложено решение, которое позволит значительно повысить эффективность управления школой иностранных языков и улучшить качество образовательного процесса.

1 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ УЧЕТА ДЕЯТЕЛЬНОСТИ ШКОЛЫ ИННОСТРАННЫХ ЯЗЫКОВ, ПРОГРАММНЫХ АНАЛОГОВ, МЕТОДОВ И АЛГОРИТМОВ РЕШЕНИЯ ПОСТАВЛЕННЫХ ЗАДАЧ

1.1 Описание деятельности школы иностранных языков

Школа иностранных языков – это учебное заведение, специализирующееся на преподавании различных иностранных языков с целью совершенствования навыков устной и письменной речи, понимания иноязычных текстов и понимания культуры. Деятельность таких школ обычно основывается на следующих принципах и задачах:

1 Базовое языковое образование: языковые школы обеспечивают качественное обучение основам выбранного языка, включая грамматику, лексику, фонетику и орфографию. На этом этапе у студентов формируются базовые навыки общения на иностранном языке.

2 Развитие коммуникативной компетенции: одной из важнейших задач школы является развитие способности свободно общаться на иностранном языке. Для этого проводятся различные уроки и практические занятия, направленные на развитие навыков аудирования, говорения, чтения и письма.

3 Подготовка к экзаменам: многие языковые школы предлагают подготовку к международным экзаменам, таким как TOEFL, IELTS и DELE на изучаемом языке. Это дает студентам официальное подтверждение их знаний и повышает их шансы на поступление в зарубежные вузы и продвижение по карьерной лестнице.

4 Персонализация обучения: языковые школы стремятся предложить индивидуальные методы обучения, учитывая уровень знаний, потребности и цели каждого студента. Для этого используются различные методы и технологии, а также индивидуальные занятия с преподавателями.

5 Приобщение к культуре страны изучаемого языка: языковые школы часто организуют мероприятия, направленные на погружение студентов в культуру страны изучаемого языка. К ним можно отнести культурные мероприятия, встречи с носителями языка и экскурсии.

Школы иностранных языков играют важную роль в развитии языковых навыков и межкультурного взаимопонимания студентов, их личностном развитии и успешной интеграции в международное сообщество.

Учет деятельности школы иностранных языков включает в себя несколько ключевых аспектов.

1 Учет учебного процесса.

Планирование курсов. Определение расписания занятий, форматов (групповые, индивидуальные) и уровней (начальный, средний, продвинутый). Разработка учебных планов и программ для различных курсов.

Учет посещаемости. Ведение журналов посещаемости для отслеживания присутствия студентов на занятиях. Анализ причин пропусков и разработка мер по их снижению.

Оценка успеваемости. Проведение регулярных тестов и контрольных работ для оценки прогресса студентов. Ведение базы данных с результатами успеваемости и обратной связью от преподавателей.

2 Финансовый учет.

Доходы. Оплата за курсы: Учет поступлений от студентов за обучение. Важно фиксировать даты оплаты, суммы и способы оплаты (наличные, банковские карты, электронные платежи). Дополнительные услуги: Учет доходов от дополнительных услуг, таких как индивидуальные занятия, подготовка к экзаменам, семинары и мастер-классы.

Расходы. Заработная плата: Учет выплат преподавателям, административному персоналу и другим сотрудникам. Аренда помещения: Фиксация расходов на аренду классов и офисных помещений. Коммунальные услуги: Учет затрат на электроэнергию, воду, интернет и другие услуги. Реклама и маркетинг: Расходы на продвижение школы, включая онлайн-рекламу, печатные материалы и мероприятия. Учебные материалы: Закупка учебников, методических пособий, канцелярских товаров.

Финансовая отчетность. Подготовка ежемесячных и годовых отчетов о доходах и расходах. Анализ финансовых показателей для оценки рентабельности и эффективности работы школы.

3 Управление кадрами.

Набор преподавателей. Процесс подбора квалифицированных специалистов: размещение вакансий, проведение собеседований, проверка рекомендаций. Оценка квалификации и опыта преподавателей.

Обучение и развитие. Организация тренингов и семинаров для повышения квалификации преподавателей. Создание системы наставничества для новых сотрудников.

Учет рабочего времени. Ведение учета рабочего времени преподавателей и административного персонала. Определение коэффициентов нагрузки для преподавателей в зависимости от количества часов занятий.

4 Маркетинг и реклама.

Анализ рынка. Исследование конкурентов и потребностей целевой аудитории. Определение уникальных предложений (USP) школы.

Рекламные кампании. Разработка рекламных материалов (брошюры, флаеры, баннеры). Использование онлайн-платформ (социальные сети, сайт школы) для привлечения студентов.

Обратная связь. Сбор отзывов от студентов о качестве обучения и уровне сервиса. Проведение опросов для улучшения учебного процесса и удовлетворенности клиентов.

5 Административный учет.

Документация. Ведение необходимых документов: договоров с учениками, актов выполненных работ, счетов-фактур. Хранение документации в соответствии с законодательными требованиями.

Управление качеством. Разработка стандартов качества обучения и обслуживания. Проведение регулярных проверок и аудитов для обеспечения соответствия стандартам.

Эффективный учет деятельности школы иностранных языков требует комплексного подхода к управлению всеми аспектами работы учреждения. Это позволяет не только оптимизировать внутренние процессы, но и повысить уровень удовлетворенности студентов, что в свою очередь способствует росту репутации школы и увеличению числа клиентов.

Ни для кого не секрет, что знание иностранного языка, а лучше нескольких, является шикарной перспективой для дальнейшей самореализации человека: профессиональной, творческой и т.п. Поэтому большинство современных родителей, понимая значимость владения иностранными языками, уделяют особое внимание тому, чтобы их дети с малых лет начали изучать иностранный язык.

На сегодняшний день существует масса языковых школ, где обучение проходят не только дети, но и взрослые [1]. На диаграмме, рисунок 1.1, представлены наиболее популярные иностранные языки для изучения.

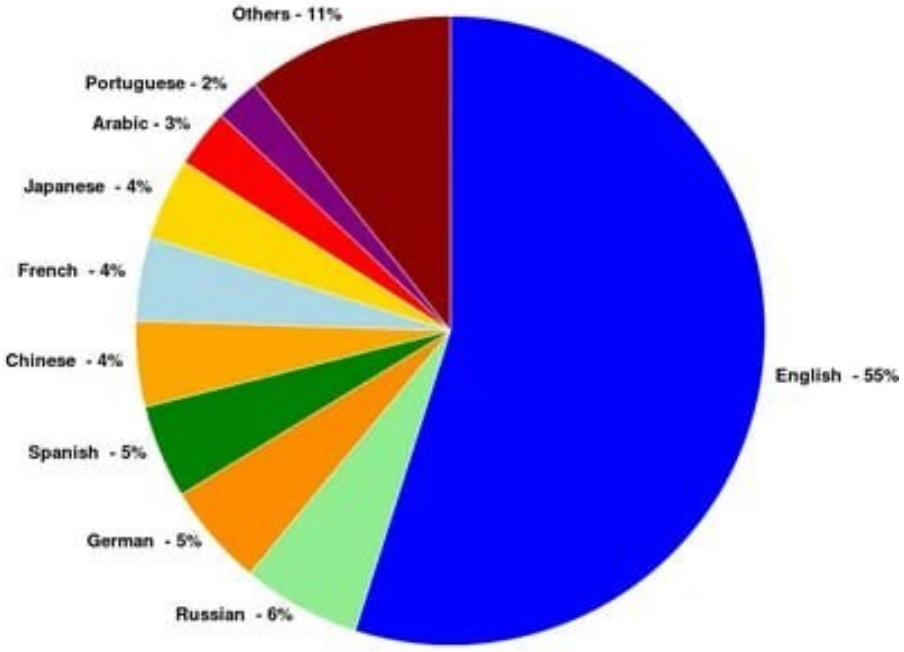


Рисунок 1.1 – Диаграмма наиболее популярных языков для изучения.

Самыми популярными языками являются английский, русский и немецкий языки. Английский и русский языки изучаются как язык международной коммуникации, а немецкий язык – для обучения в техникумах и университетах.

На рисунке 1.2 показана польза от изучения иностранного языка.



Рисунок 1.2 – Польза от изучения иностранного языка

Помимо того, что иностранный язык помогает в карьерной самореализации, необходим для обучения в иностранных учебных заведениях и трудоустройства за границей, знание иностранного языка способствует развитию мозга человека.

1.2 Обзор программных аналогов автоматизированной системы учета деятельности школы иностранных языков

Для учета деятельности школы иностранных языков можно рассмотреть три популярных программных решения и выявить их недостатки, которые могут быть устранены в разработанной программе.

1 Moodle. Рисунок 1.3.



Рисунок 1.3 – Система дистанционного обучения Moodle

Moodle – многофункциональная СДО со всеми возможностями для обучения и аттестации слушателей курса. Условно-бесплатная версия подойдет тем, у кого есть технические навыки или есть возможность найма соответствующих специалистов. Кроме того, пользователям серверной версии нужно знать, что запустить курс быстро не получится. Установка Moodle может занять больше месяца.

С облачной версией работать проще. Серьезные технические навыки не нужны, а за работоспособность системы отвечают разработчики Moodle. Минус этой СДО в том, что она платная, но при этом может оказаться недостаточно удобной для пользователей. Например, из-за хранения информации на иностранном сервере. Или недостаточного количества шаблонов, сложного интерфейса [2].

2 CRM-система “Bitrix24”. Рисунок 1.4.

Программа Битрикс 24 объединила современные разработки в области взаимодействия с клиентами и ведения аналитики. Это гибкая и эффективная CRM (расшифровывается как Customer Relationship Management) – система, помогающая контролировать все каналы коммуникации с заказчиками и продажи. Один сервис позволяет собирать и хранить данные о клиентах, отслеживать работу менеджеров, анализировать эффективность разных каналов рекламы и решать десятки других задач [3].



Рисунок 1.4 – CRM-система “Bitrix24”

Недостатки. Избыточный функционал, не всегда необходимый для школ иностранных языков. Сложности с настройкой и адаптацией под специфические нужды образовательных учреждений. Ограниченная поддержка специфических образовательных процессов.

3 Учебный центр "1С:Образование". Рисунок 1.5.

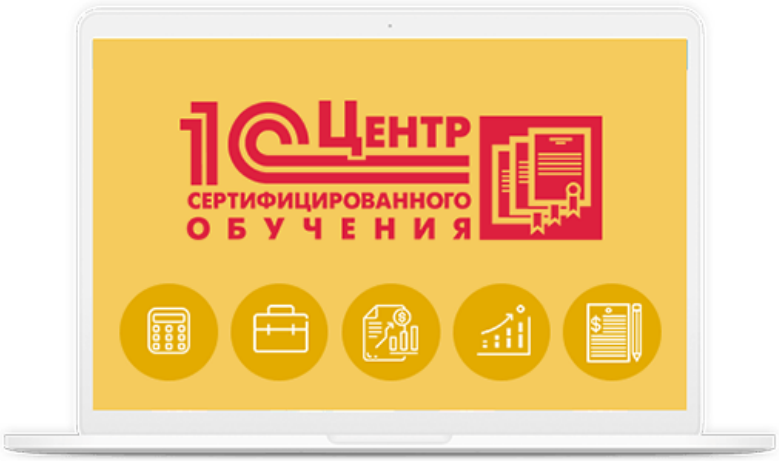


Рисунок 1.5 – Учебный центр "1С:Образование"

Учебный центр "1С:Образование" – это комплексное решение для автоматизации образовательного процесса в школах, колледжах и вузах. Основное назначение системы – организация обучения с использованием электронных образовательных ресурсов. Учителя, учащиеся и родители могут работать с системой в локальной сети школы или дистанционно через интернет. С системой можно работать как при помощи обычных компьютеров и ноутбуков, так и мобильных устройств – планшетов и смартфонов [4].

Недостатки:

– сложность интерфейса: требует времени на обучение и привыкание;

– высокая стоимость: лицензии и обновления могут быть дорогими;

– ограниченные возможности интеграции: не всегда легко интегрируется с другими системами.

Создание программы для учета деятельности школы иностранных языков требует внимательного подхода к существующим решениям. Изучение недостатков программ позволит разработать более эффективный и удобный инструмент, который окажется полезным как для преподавателей, так и для учащихся.

1.3 Обзор методов и алгоритмов решения поставленных задач автоматизированной системы учета деятельности школы иностранных языков

Для реализации автоматизированной системы учета деятельности школы иностранных языков используются современные методы и алгоритмы, направленные на удовлетворение ключевых критериев, обеспечивающих эффективность и удобство для пользователей.

В первую очередь, это простота. Приложение не должно содержать излишние или сложные функции, которые могут усложнить работу и запутать пользователей. Интерфейс должен быть интуитивно понятным и лаконичным. Применение адаптивного дизайна для того, чтобы приложение было удобным как на настольных компьютерах, так и на мобильных устройствах. Создание сценариев пользовательского взаимодействия для минимизации количества действий, необходимых для выполнения базовых операций.

Еще один важный критерий – понятность. Ученики и преподаватели школы иностранных языков не обязаны быть экспертами в компьютерных технологиях, поэтому приложение должно быть максимально удобным и простым в использовании. Программное обеспечение должно предлагать понятный пользовательский опыт, позволяющий любому человеку быстро освоить основные функции.

Третий критерий – безопасность. Для хранения данных пользователей и курсов применяется реляционная база данных, например, MySQL или PostgreSQL. Обеспечение безопасной авторизации с использованием алгоритмов хэширования паролей. Ограничение доступа к функциям в зависимости от роли пользователя (студент, преподаватель, администратор). Регулярный аудит безопасности системы.

Четвертый критерий – отказоустойчивость. Приложение должно быть устойчиво к сбоям и иметь механизмы защиты от ввода некорректных данных. Используются алгоритмы валидации данных для проверки корректности вводимой информации (например, проверка форматов email и телефона). Это особенно важно, учитывая, что пользователи могут случайно ввести неправильную информацию. Система должна справляться с такими ситуациями без остановки работы.

Помимо основных критериев, приложение для школы иностранных языков должно обладать набором основных функций, необходимых для обеспечения ключевых бизнес-процессов. Во-первых, это авторизация и регистрация, которые позволяют новым пользователям создавать учетные записи и входить в систему. Пользователи должны иметь возможность выбрать тариф занятий и записаться на разные курсы.

Администрация школы иностранных языков должна иметь функции управления данными пользователей и функции для работы с данными курсов.

Соблюдение всех этих критериев и предоставление функционала помогут создать удобное, стабильное и полезное приложение для школы иностранных языков, которое будет способствовать успешной организации работы школы и удовлетворять потребности ее пользователей.

2 ФУНКЦИОНАЛЬНОЕ МОДЕЛИРОВАНИЕ НА ОСНОВЕ СТАНДАРТА IDEF0

IDEF0 является нотацией графического моделирования, используемая для создания функциональной модели, отображающей структуру и функции системы, а также потоки информации и материальных объектов, связывающих эти функции.

При создании программного продукта для начала необходимо понять, как работает система, которую нужно автоматизировать. Необходимо построить модель, которая будет описывать всю работу системы, опираясь на знания всех участников бизнес-процесса.

На рисунке 2.1 представлена контекстная диаграмма верхнего уровня. Входными данными являются заявка на занятие, актуальная информация о расписании и всевозможные необходимые данные о студентах и преподавателях. В результате процесса на выходе мы получаем заполненное расписание преподавателей и систему управления доступом различных ролей (преподавателей, учеников, администраторов).

Для контроля и управления процессом используются политика безопасности данных, система мониторинга эффективности обучения и используемые учебные материалы.

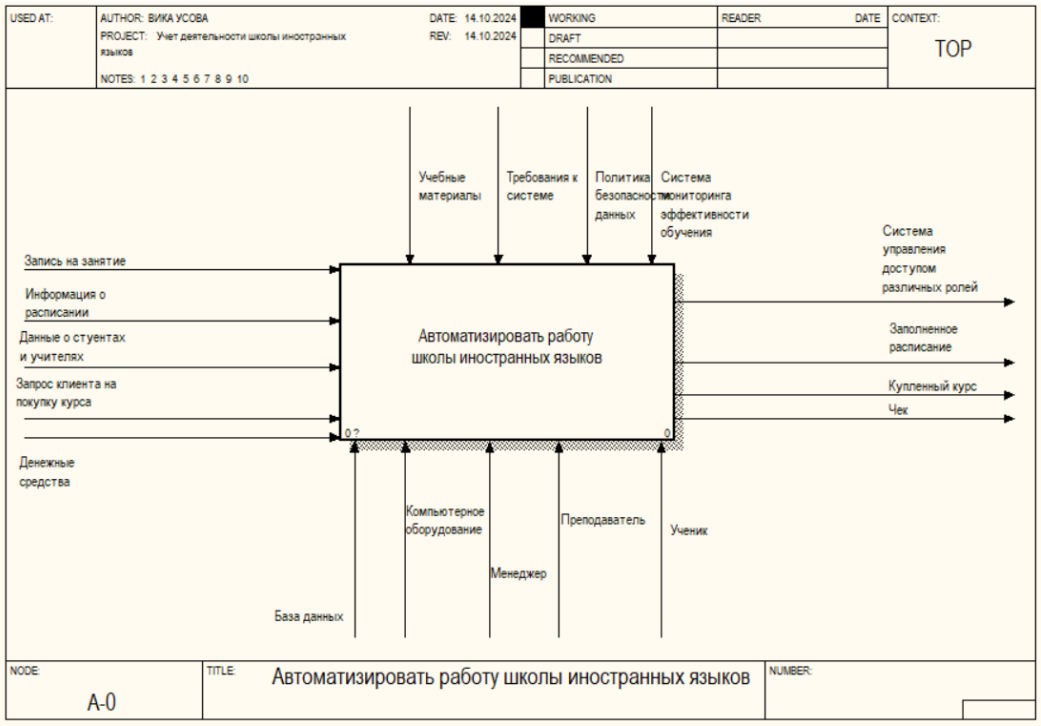


Рисунок 2.1 – Контекстная диаграмма верхнего уровня

Ресурсами, необходимыми для выполнения процесса являются:

– компьютерное оборудование, с помощью которого происходит работа данного процесса;

– база хранения всей информации, касающейся уроков: времени, преподавателей и учащихся;

– менеджер, осуществляющий контроль за своевременной оплатой занятий, а также обеспечивающий обработку обратной связи;

– преподаватель, проводящий занятия;

– ученик.

Входными параметрами являются данные о преподавателях и студентах, информация о расписании, запрос клиента на покупку курса, запись на занятие, денежные средства. Выходные параметры – купленный курс, заполненное расписание, система управления доступом различных ролей, чек. Управляющими параметрами служат учебные материалы, требования к системе, политика безопасности данных, система мониторинга эффективности обучения.

Декомпозиция контекстной диаграммы, показанная на рисунке 2.2, иллюстрирует разделение основного процесса на компоненты-задачи.

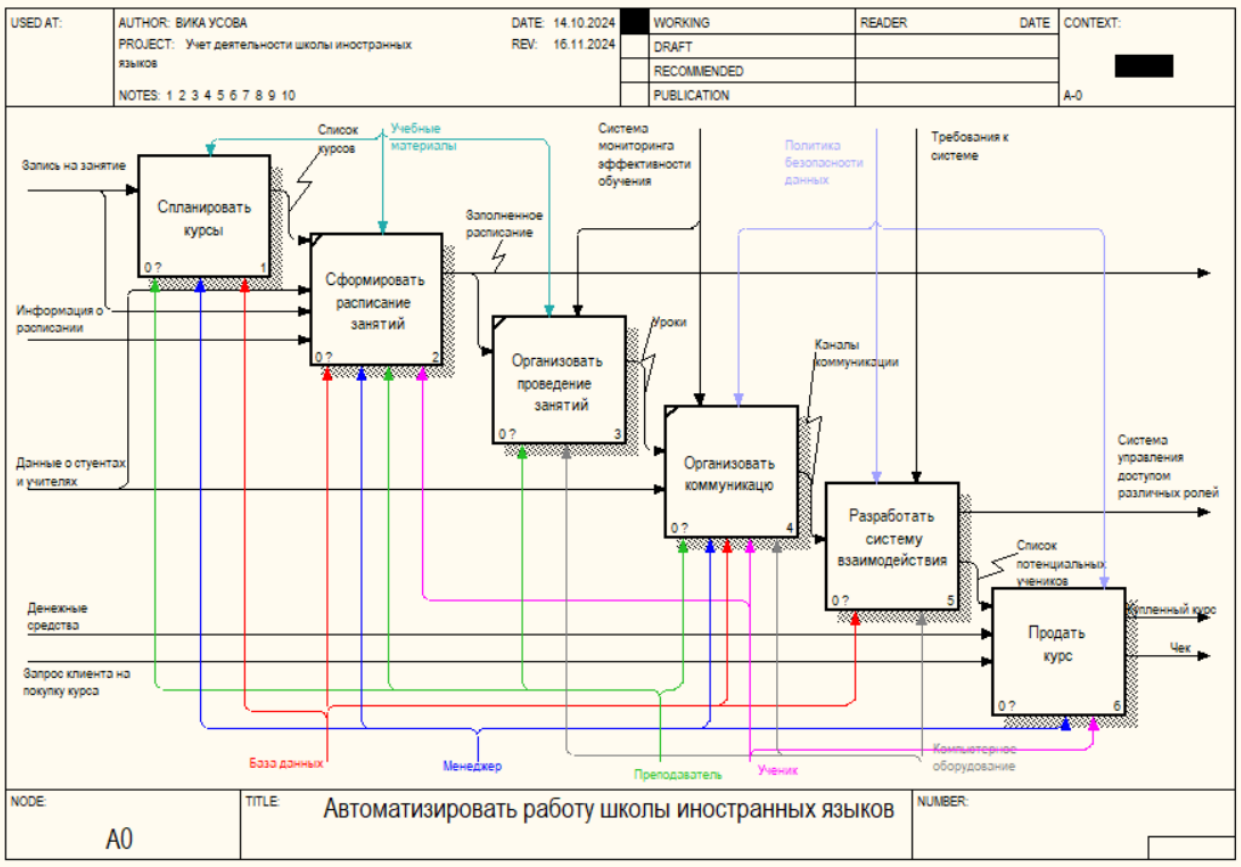


Рисунок 2.2 – Декомпозиция контекстной диаграммы верхнего уровня

Декомпозиция нужна для лучшего понимания бизнес-процесса. На рисунке 2.2 показано, как различные функции связаны между собой, какие данные необходимы для выполнения каждой функции и как данные передаются между функциями. Это помогает оптимизировать процесс учета деятельности школы иностранных языков.

В общем виде процесс учета деятельности школы иностранных языков можно разделить на пять подпроцессов:

– спланировать курсы;
– сформировать расписание занятий;
– организовать проведение занятий;
– организовать коммуникацию между участниками;
– разработать систему взаимодействия клиентов и администрации;
– продать курс.

В свою очередь подпроцесс «Спланировать курсы» может быть разбит еще на несколько этапов. Декомпозиция представлена на рисунке 2.3.

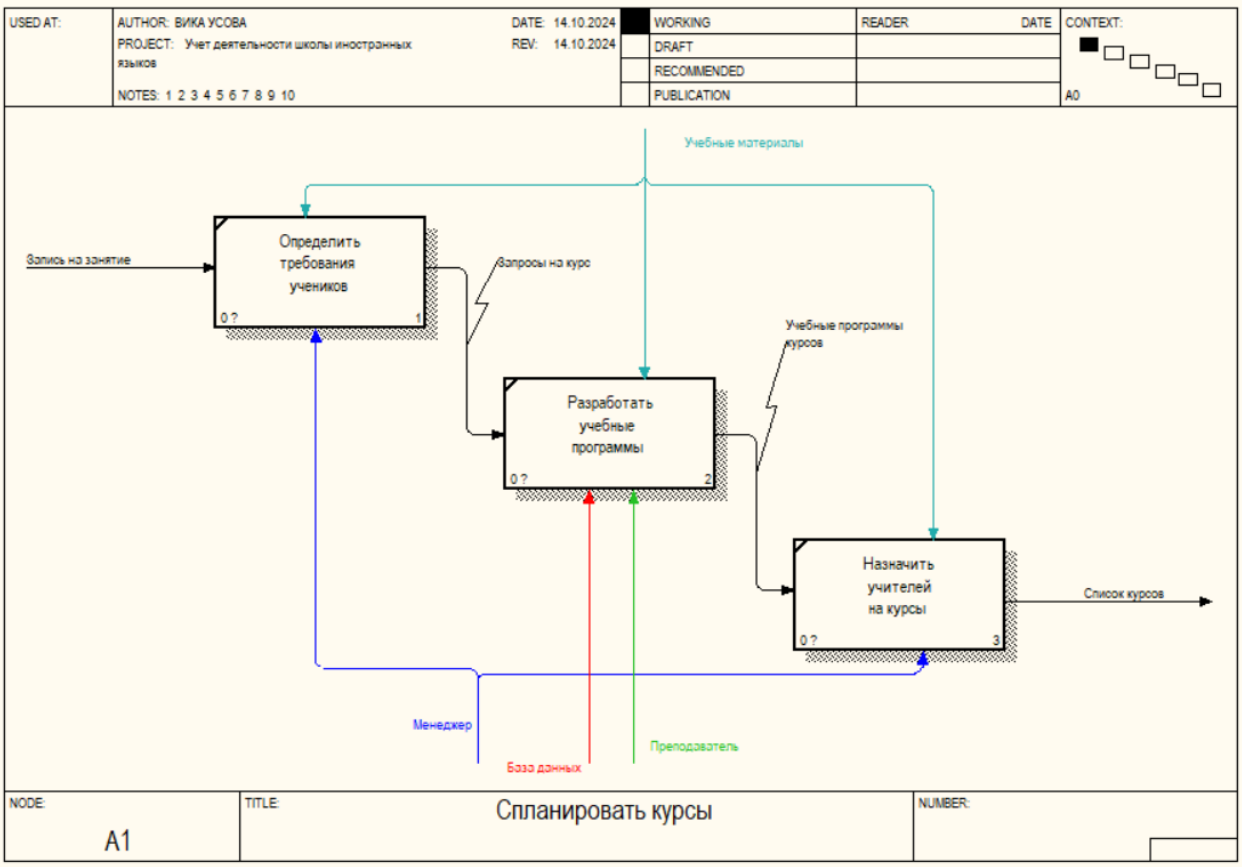


Рисунок 2.3 – Декомпозиция блока «Спланировать курсы»

Цель блока «Спланировать курсы» заключается в разработки и организации курсов иностранных языков с учетом потребностей целевой аудитории и ресурсов учебного заведения. Для достижения цели менеджер

должен определить требования учеников и назначить преподавателей на курсы. Преподаватели в свою очередь должны разработать учебные программы.

Декомпозиция блока «Разработать систему взаимодействия» проиллюстрирована на рисунке 2.4.

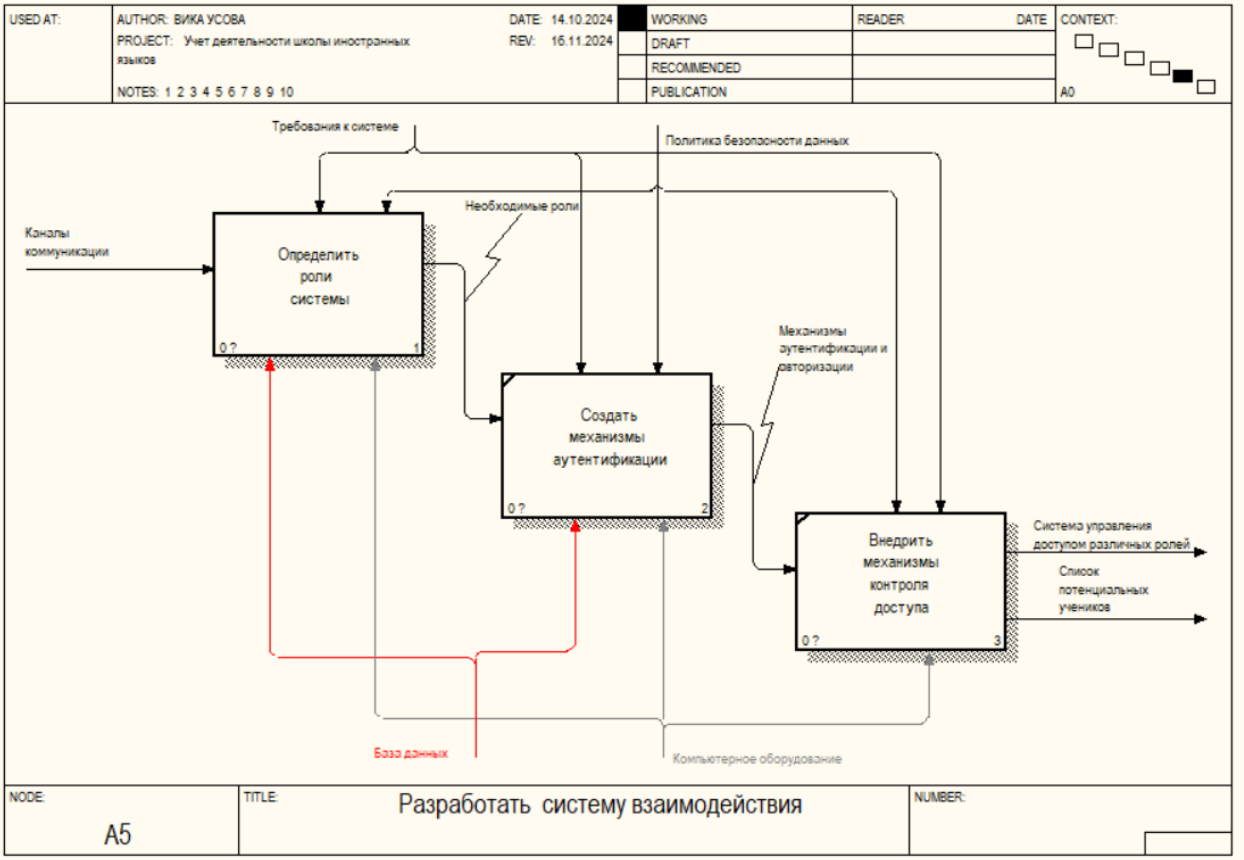


Рисунок 2.4 – Декомпозиция блока «Разработать систему взаимодействия»

Целью данного блока является создание эффективной системы, обеспечивающей оперативное и удобное взаимодействие между клиентами и администрацией школы. Данный блок можно разделить на следующие этапы:

– определить роли системы;

– создать механизмы аутентификации;

– внедрить механизмы контроля доступа к конфиденциальной информации.

Декомпозиция блока «Определить роли системы» представлена на рисунке 2.5.

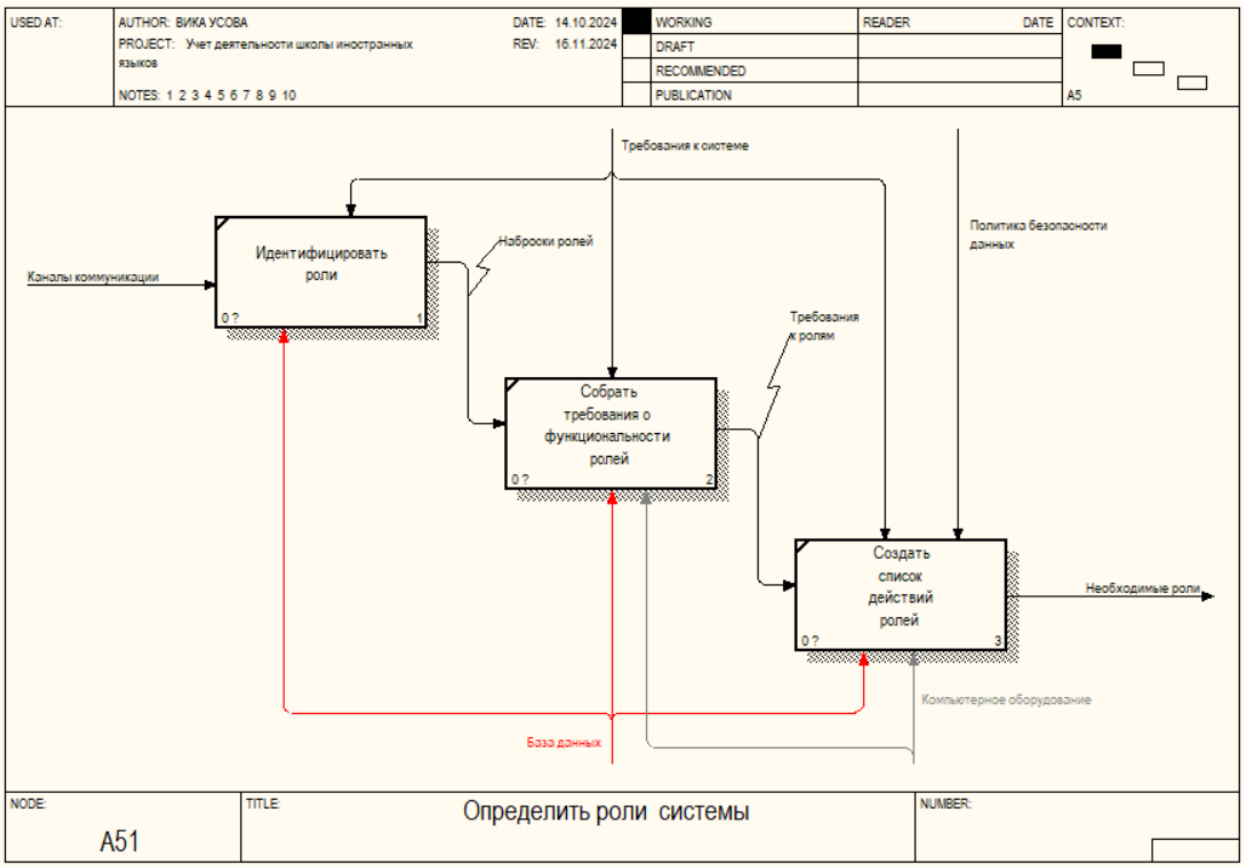


Рисунок 2.5 – Декомпозиция блока «Определить роли системы»

Итогом данного блока является установление четких ролей и обязанностей пользователей системы для обеспечения эффективного взаимодействия и выполнения задач.

Процесс продажи курса также можно разделить на 4 этапа. Декомпозиция блока «Продажа курса» представлена на рисунке 2.6.

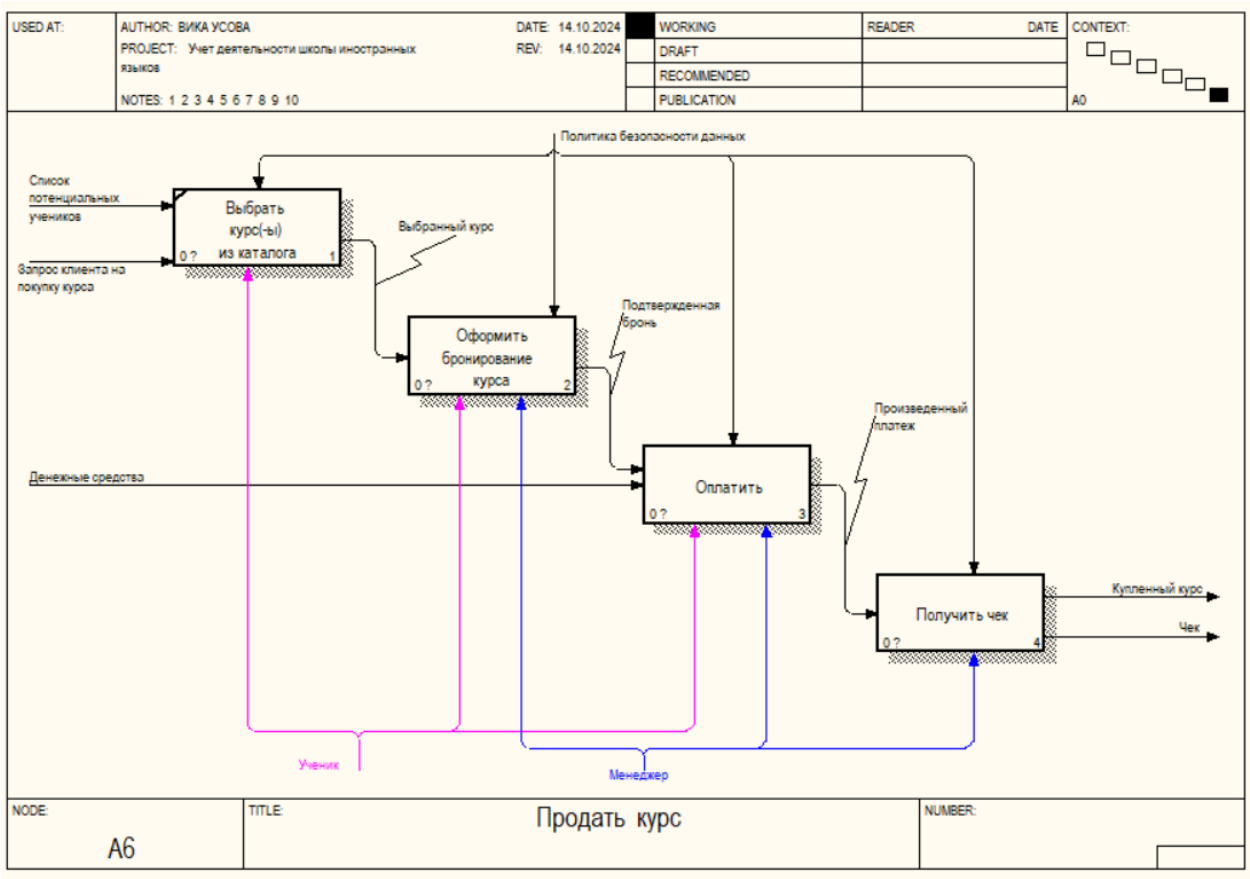


Рисунок 2.6 – Декомпозиция блока «Продать курс»

Добросовестная реализация процесса продажи курсов необходима для обеспечения дохода и удовлетворения потребностей клиентов. Эффективная продажа курсов требует постоянного анализа рынка и адаптации предложений в соответствии с изменениями спроса. Важно обеспечить высокий уровень сервиса на всех этапах взаимодействия с клиентами для формирования положительного имиджа организации.

В свою очередь подпроцесс «Оформить бронирование курса» может быть разбит еще на несколько этапов (декомпозиция представлена на рисунке 2.7):

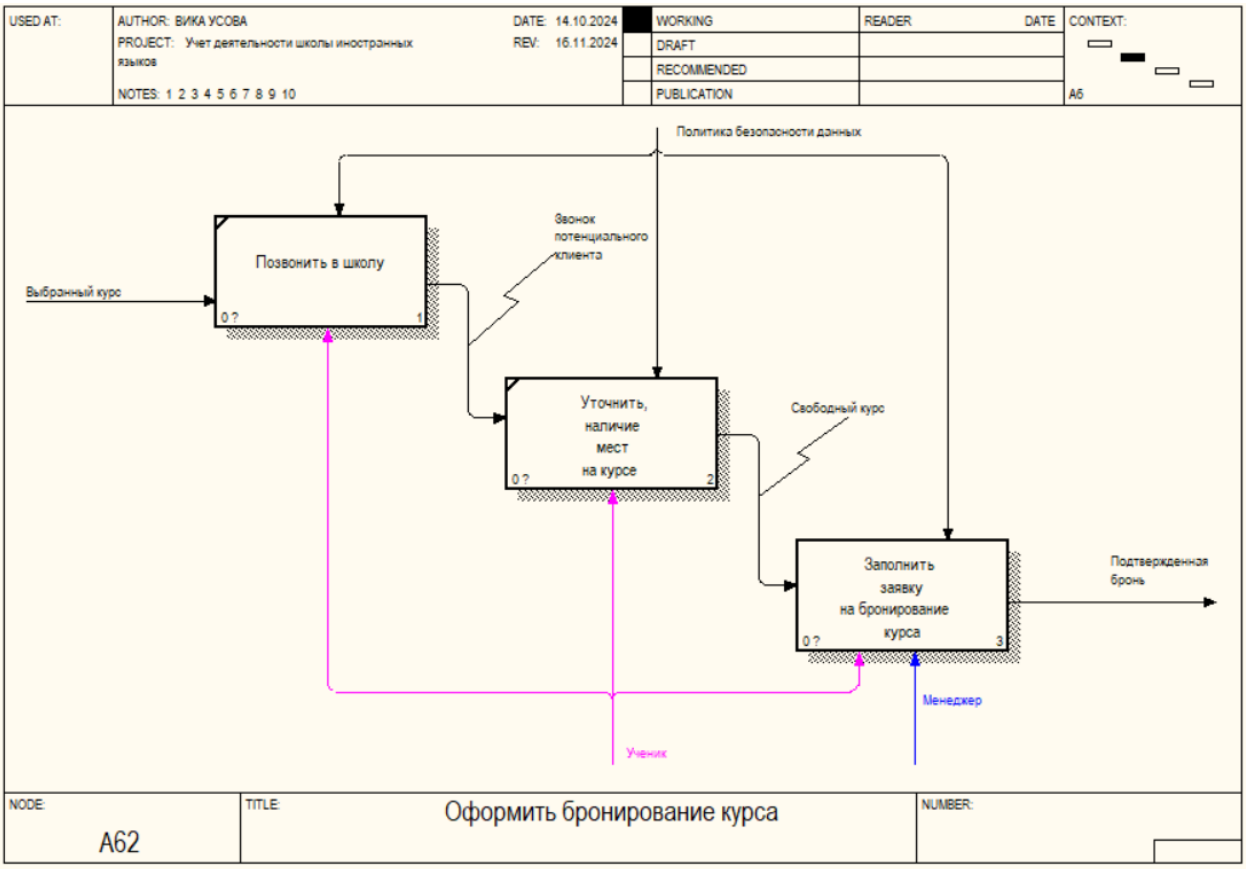


Рисунок 2.7 – Декомпозиция блока «Оформить бронирование курса»

Необходимо обеспечить простой и удобный процесс бронирования курсов для клиентов с минимальными временными затратами. Важно обеспечить безопасность личной информации клиентов и защиту данных в процессе бронирования. Удобство и простота процесса бронирования напрямую влияют на уровень удовлетворенности клиентов и их желание вернуться.

В свою очередь подпроцесс «Заполнить заявку на бронирование курса» можно также разделить на 3 подпроцесса. Декомпозиция такого блока представлена на рисунке 2.8.

Удобный процесс заполнения заявки способствует повышению уровня удовлетворенности клиентов и увеличивает вероятность бронирования курса. Необходимо обеспечить защиту личной информации клиентов и соблюдение норм конфиденциальности в процессе обработки заявок.

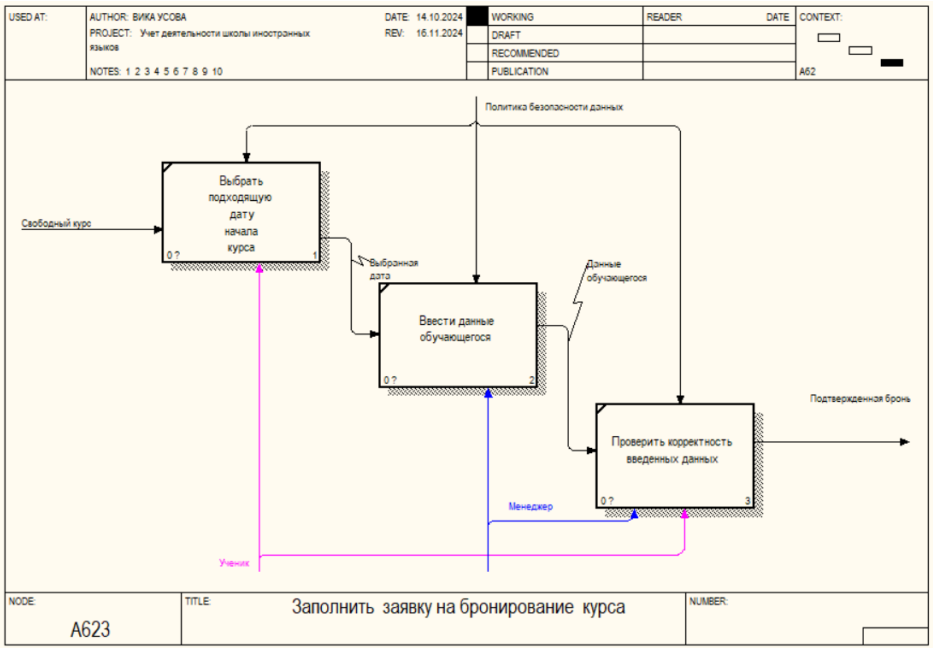


Рисунок 2.8 – Декомпозиция блока «Заполнить заявку на бронирование курса»

Блок «Оплатить» можно разбить на 3 подпроцесса. Декомпозиция представлена на рисунке 2.9.

Необходимо обеспечить безопасный и удобный процесс оплаты курсов для клиентов, забронировавших курс для получить доступ к обучению. Удобство и простота процесса оплаты напрямую влияют на уровень удовлетворенности клиентов и их желание повторно воспользоваться услугами.

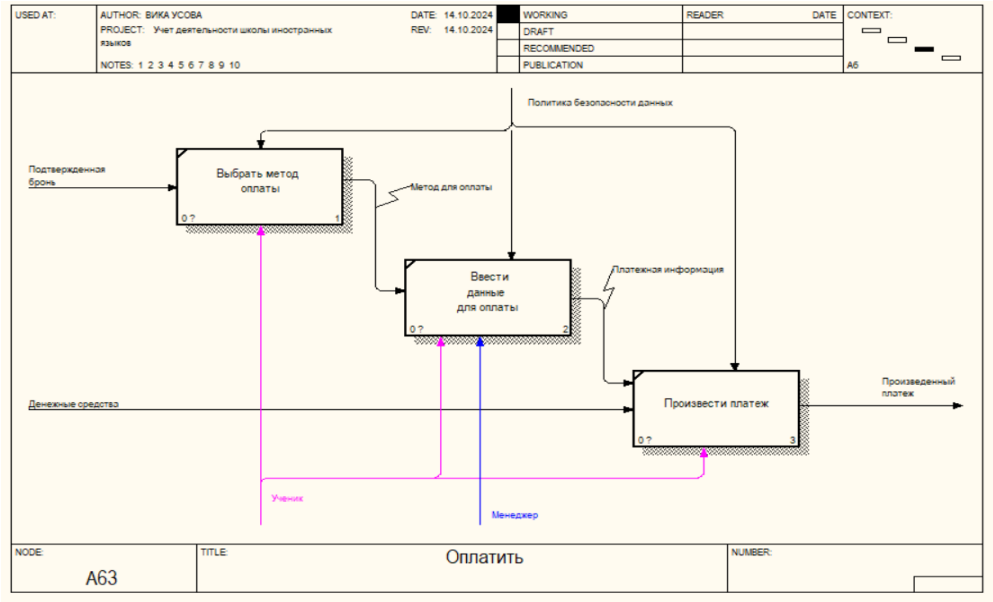


Рисунок 2.9 – Декомпозиция блока «Оплатить»

Блок «Получить чек» можно разбить на 2 подпроцесса. Декомпозиция представлена на рисунке 2.10.

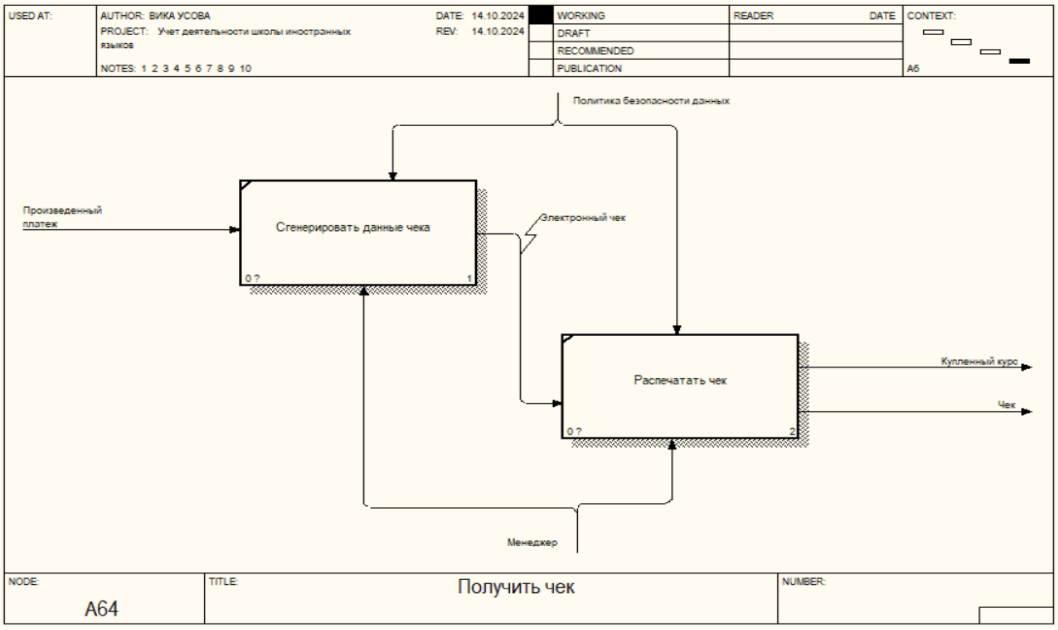


Рисунок 2.10 – Декомпозиция блока «Получить чек»

Таким образом, функциональная модель позволяет подробно ознакомиться с внутренними процессами деятельности школы иностранных языков. Она способствует повышению эффективности, улучшению управления и обеспечивает основу для дальнейшего развития и оптимизации бизнес-процессов. Анализ процесса бронирования в нотации IDEF0 дает возможность более эффективного управления и адаптации к изменениям в бизнес-среде. Определение ключевых активностей и связей между ними помогает выявить узкие места и возможности для оптимизации процесса [5].

3 СТРУКТУРА ИСПОЛЬЗУЕМЫХ ДАННЫХ

Для описания структуры используемых данных применяется ER-диаграмма, которая представляет собой графическое отображение архитектуры базы данных и служит для моделирования данных. В ней показано, как различные сущности взаимосвязаны между собой. ER-диаграммы являются важным инструментом при проектировании реляционных баз данных, так как помогают визуализировать, как данные будут храниться и взаимодействовать внутри системы.

ER-диаграмма данных, которые используются в автоматизированной системе учета деятельности школы иностранных языков, представлена на рисунке 3.1.

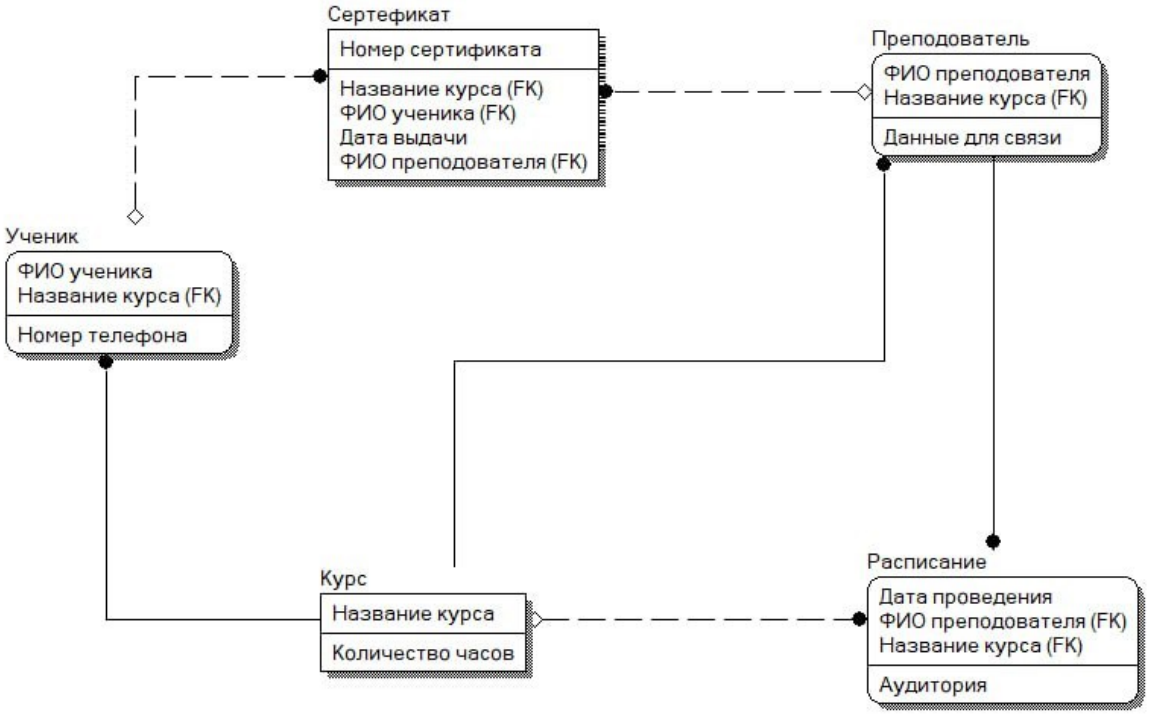


Рисунок 3.1 – ER-диаграмма

На данной диаграмме представлены различные сущности.

Ученик. В данной сущности представлены такие атрибуты как ФИО ученика, название курса и номер телефона. Атрибут ФИО ученика имеет тип данных string и служит для связи с сущностью сертификат. Название курса, также тип данных string, в свою очередь служит для связи с сущностью Курс.

Сертификат. В данной сущности представлены такие атрибуты как Номер сертификата (тип данных int), Название курса (тип данных string), ФИО ученика(тип данных string), Дата выдачи (тип данных Date), ФИО

преподавателя (тип данных string). Атрибут ФИО преподавателя служит для связи с сущностью Преподаватель.

Преподаватель. Данная сущность включает в себя такие атрибуты как ФИО преподавателя (тип данных string), Название курса (тип данных string), Данные для связи (тип данных string). По атрибуту ФИО Преподавателя данная сущность связана с сущностью Расписание, по Названию курса – с сущностью Курс.

Расписание. Сущность включает в себя атрибуты: Дата проведения (тип данных Date), ФИО преподавателя (тип данных string), Название курса (тип данных string).

Курс. В данную сущность включены такие атрибуты как Название курса (тип данных string) и Количество часов (тип данных double).

ER-диаграмма (диаграмма сущностей и связей) – это инструмент, используемый в процессе проектирования баз данных.

ER-диаграмма помогает наглядно представить, какие сущности (таблицы) существуют в базе данных и как они связаны между собой. Позволяет четко определить, какие данные будут храниться, а также их характеристики (атрибуты). Помогает разработчикам и аналитикам в процессе проектирования, облегчая понимание требований и взаимодействия между различными частями системы. Позволяет выявить потенциальные проблемы в структуре данных до начала реализации, что может сэкономить время и ресурсы. Служит хорошим средством для документирования структуры базы данных, что полезно для будущих разработчиков и администраторов. ER-диаграммы могут быть преобразованы в реляционные схемы, что упрощает создание и управление базами данных [6].

Использование ER-диаграмм делает процесс проектирования баз данных более структурированным и понятным, что в конечном итоге приводит к более эффективным и надежным системам.

4 ОПИСАНИЕ СОЗДАННЫХ ПРОГРАММНЫХ КОНСТРУКЦИЙ

При проектировании программного обеспечения важно уделять внимание визуализации взаимосвязей между различными блоками и компонентами системы. Для этого используется диаграмма компонентов, которая позволяет наглядно представить структуру системы и проанализировать взаимодействие ее элементов. С помощью такой диаграммы можно получить четкое представление о том, как компоненты системы работают совместно, что особенно важно для обеспечения стабильной и корректной работы автоматизированной системы учета деятельности школы иностранных языков.

Использование диаграммы компонентов способствует более глубокому пониманию структуры и логики работы автоматизированной системы, что, в свою очередь, повышает эффективность ее разработки и эксплуатации. Она помогает визуализировать структуру сложных систем и управлять зависимостями между их элементами. Она упрощает проектирование больших систем, позволяя разработчикам четко видеть взаимодействие компонентов и их интерфейсы.

Пример диаграммы компонентов для системы учета деятельности школы иностранных языков представлен на рисунке 4.1.

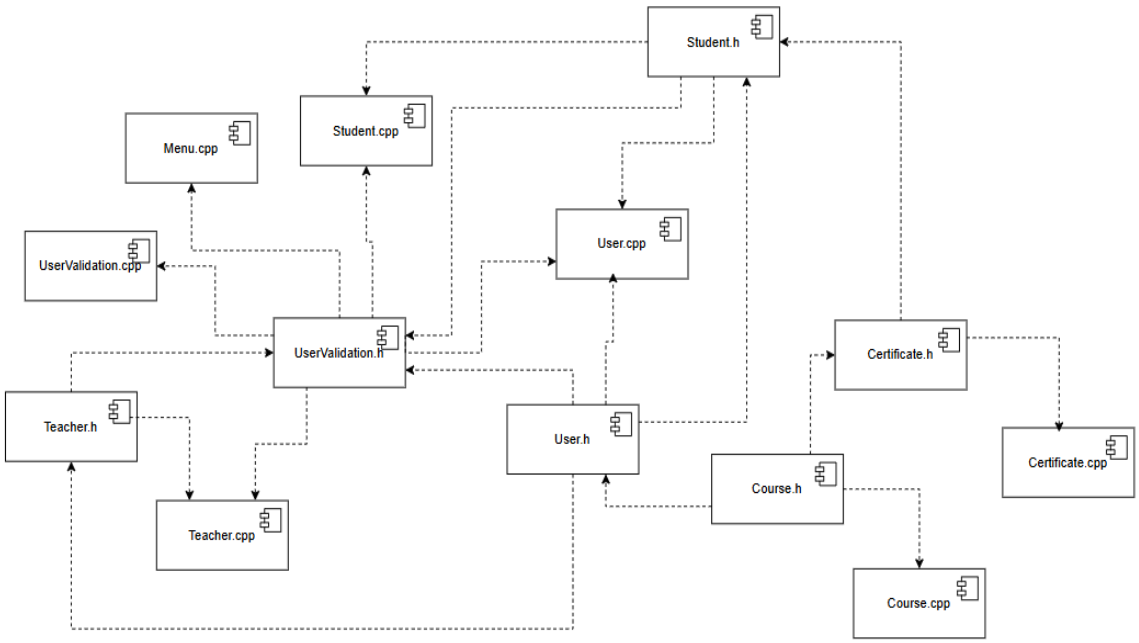


Рисунок 4.1 – Диаграмма компонентов автоматизированной системы учета деятельности школы иностранных языков

Диаграмма компонентов, отражающая структуру системы, включает в себя основные модули и взаимосвязи между ними. Она построена на основе следующих файлов кода и заголовочных файлов.

1 Certificate.cpp, Certificate.h – управляет созданием и хранением сертификатов для студентов.

2 Course.cpp, Course.h – представляет курс, включая его название, описание и связанные сертификаты.

3 Student.cpp, Student.h – хранит информацию о студентах, их успеваемости и курсах, которые они прошли.

4 Teacher.cpp, Teacher.h – представляет преподавателей, которые ведут курсы и взаимодействуют со студентами.

5 User.cpp, User.h. – общий класс для пользователей системы (студенты и преподаватели).

6 UserValidation.cpp, UserValidation.h – обеспечивает проверку данных пользователей для аутентификации и авторизации.

7 main.cpp – точка входа в программу, которая инициализирует систему и управляет основным потоком выполнения.

Эта структура позволяет организовать систему, обеспечивая четкое разделение ответственности между различными компонентами.

5 РАЗРАБОТКА И ОПИСАНИЕ ДИАГРАММЫ КЛАССОВ

В этой главе рассматриваем диаграммы классов, которые были использованы при создании автоматизированной системы учета деятельности школы иностранных языков.

В системе учета деятельности школы иностранных языков выделяются два основных класса: Schedule и Course, а также два вспомогательных перечисления: DayOfWeek и Language. Эти компоненты взаимодействуют между собой для моделирования учебного процесса.

Рассмотрим рисунок 5.1.

Класс Schedule отвечает за хранение информации о расписании занятий, включая дни недели, время и кабинет, где проходят занятия.

В классе присутствуют функциональные методы:

– displaySchedule() – отображает расписание в удобном для пользователя виде.

– static getDayAsString(DayOfWeek day) – преобразует значение перечисления DayOfWeek в строку (например, Monday в "Понедельник").

Класс Course представляет собой модель курса, включающую информацию о его характеристиках, преподавателе, студентах, языке обучения и расписании.

В данном классе присутствуют функциональные методы:

– addStudent(int studentID) – добавляет студента в список записанных на курс;

– removeStudent(int studentID) – удаляет студента из списка;

– incrementCompletedTopics() – увеличивает счетчик завершенных тем;

– markAsCompleted() – помечает курс как завершенный.

И статические методы:

– generateID() – генерирует уникальный идентификатор курса.

Класс Course включает объект класса Schedule для хранения расписания курса.

Перечисления DayOfWeek и Language используются как типы данных в соответствующих атрибутах классов Schedule и Course.

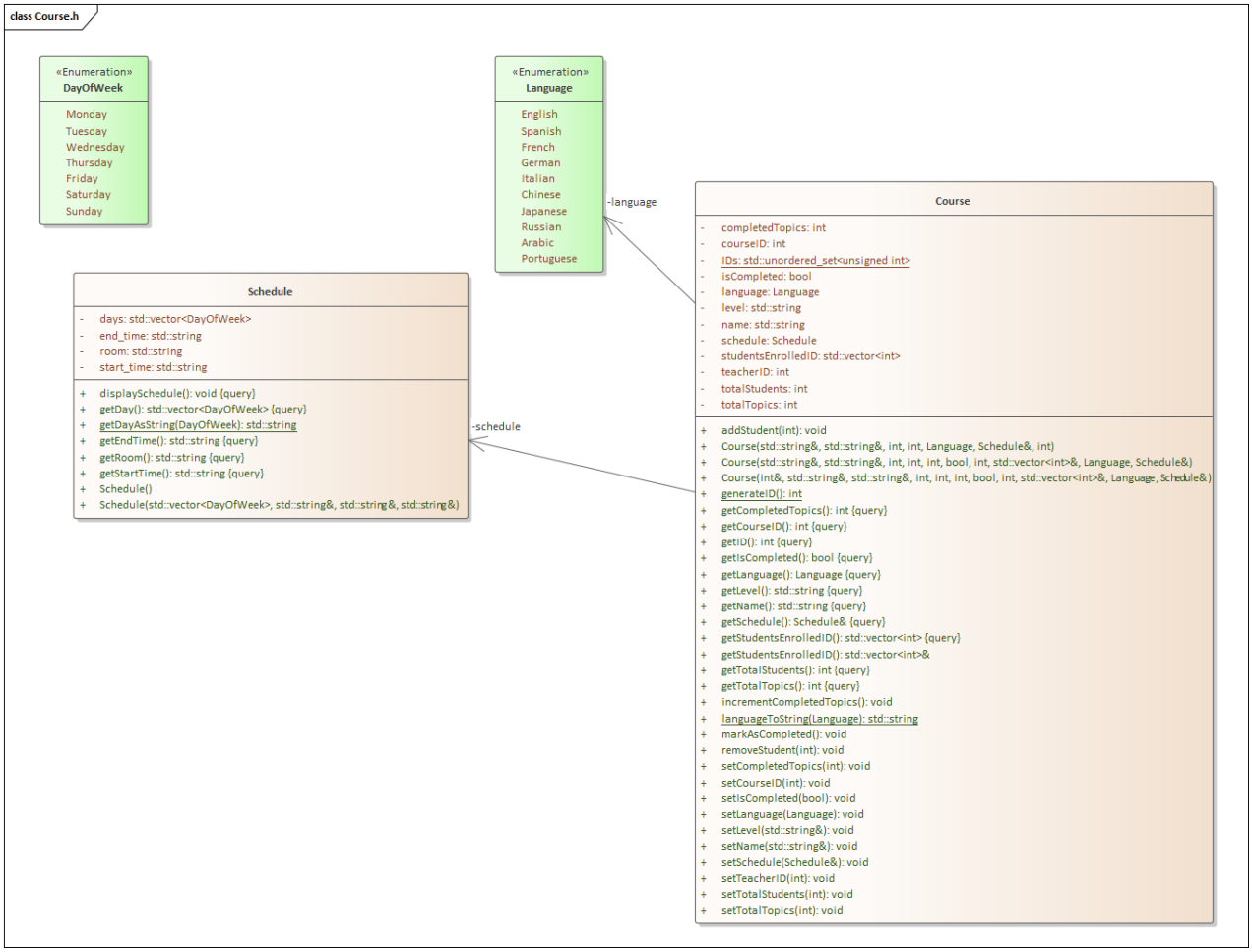


Рисунок 5.1 – Диаграмма классов, часть 1

Класс Certificate (рисунок 5.2) представляет сертификат, выдаваемый по завершению курса, и включает информацию о курсе, преподавателе и языке обучения.

В классе Certificate используются такие функциональные методы:

– static languageToString(Language language). Преобразует язык из перечисления Language в строковое представление (например, Language::English → "English");

– generateCertificateText(). Генерирует текст сертификата, включая информацию о курсе, преподавателе и уровне языка. Пример текста: "Сертификат подтверждает, что курс 'English for Beginners' (уровень A1), проведенный преподавателем Иванов Иван Иванович, успешно завершен.";

– displayCertificate(). Отображает информацию о сертификате в формате, удобном для пользователя.

Класс Certificate тесно связан с классом Course, так как атрибуты сертификата (courseID, courseName, language, languageLevel) соответствуют данным курса;

Перечисление `Language`, использованное в атрибутах language, согласовано с общими стандартами языка обучения в системе.

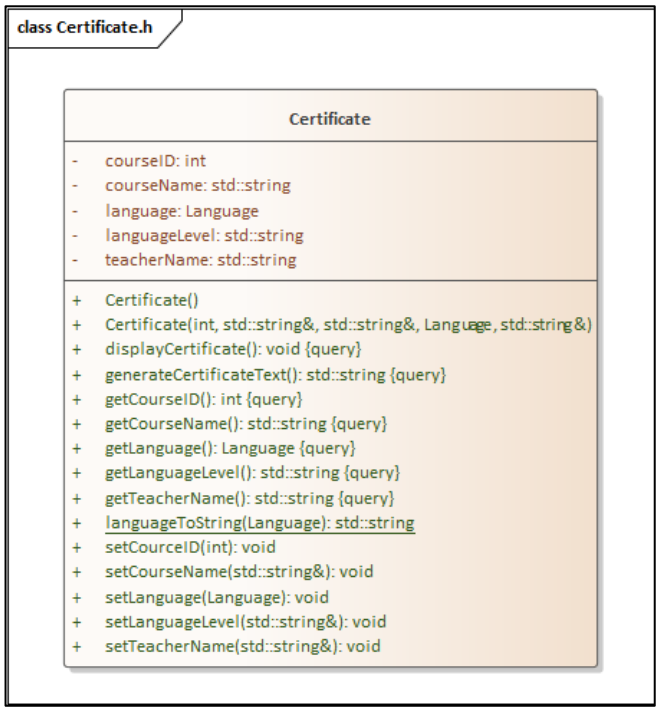


Рисунок 5.2 – Диаграмма классов, часть 2

Классы Student и Request (рисунок 5.3) используются для управления студентами и их запросами на курсы в системе учета школы иностранных языков.

Класс Student описывает информацию о студенте, включая его личные данные, зарегистрированные курсы, сертификаты и взаимодействие с системой запросов.

В качестве функциональных методов можно рассмотреть:

– hasCourse(int courseID) – проверяет, записан ли студент на указанный курс;

– generateID() – генерирует уникальный идентификатор студента;

– removeCourse(int courseID) – удаляет курс из списка студента;

– openStudentMenu() – открывает меню студента для управления его профилем, курсами и сертификатами;

– changePassword() – позволяет студенту сменить пароль;

– displayCourseTable() – отображает курсы с форматированием;

– sortCourses() – выполняет сортировку списка курсов по заданной категории;

– viewAvailableCourses() – отображает доступные курсы с сортировкой;

– findCourse() – ищет курс по названию или другим параметрам;

– enrollInCourse() – создает запрос на запись студента на курс;

– viewMyCourses() – отображает курсы, на которые записан студент;

– viewSchedule() – показывает расписание курсов студента;

– viewRequests() – отображает список запросов студента;

– viewCertificates() – отображает полученные сертификаты;

– editProfile() – позволяет редактировать профиль студента.

Класс Request описывает запрос студента на запись на курс.

Функциональный метод static statusToString(RequestStatus status). Преобразует статус из перечисления RequestStatus в строку (например, Expected → "Ожидается").

Student и Request. Каждый объект Student содержит список объектов Request, представляющий запросы студента на запись на курсы.

Student и Course. Идентификаторы курсов хранятся в атрибуте coursesID, а также используются для управления сертификатами и расписанием.

Request и Course. Поле courseID в Request связывает запрос с конкретным курсом.

Классы Student и Request обеспечивают полный цикл управления информацией о студентах, их курсах и запросах, что делает систему гибкой и удобной для пользователей.

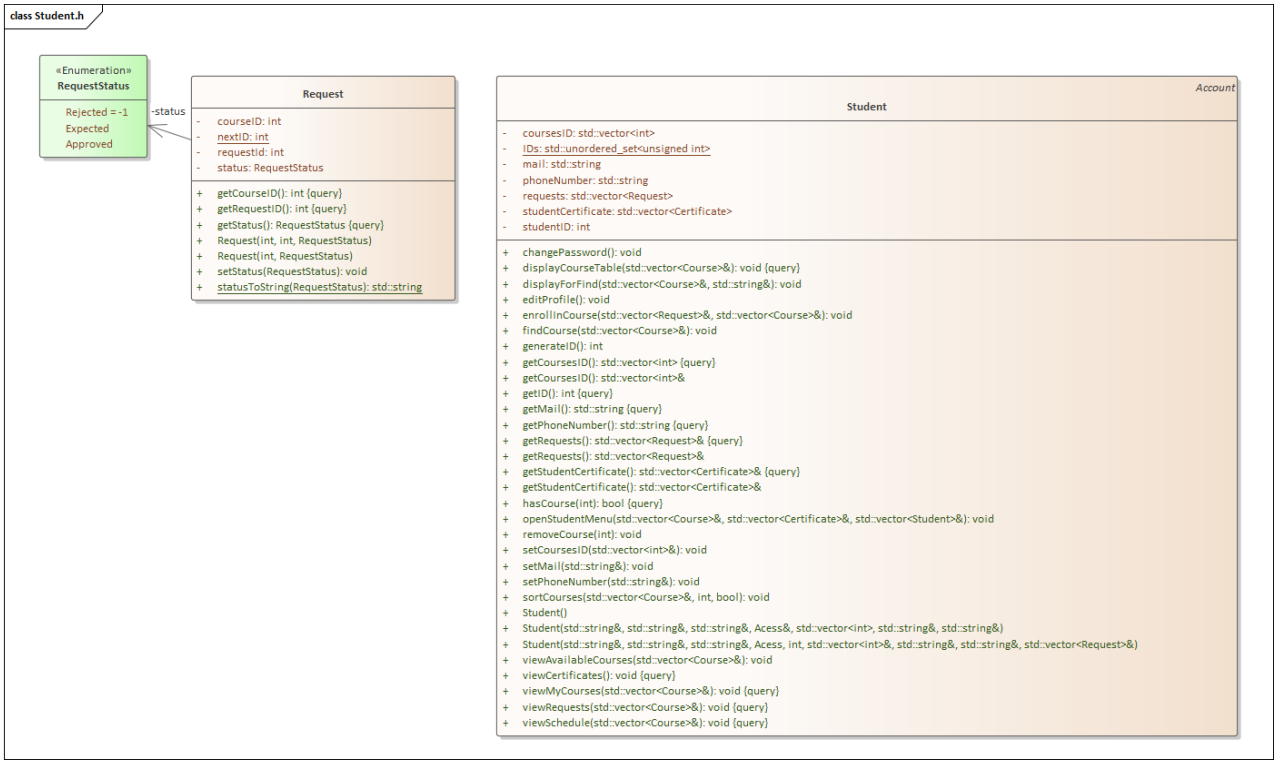


Рисунок 5.3 – Диаграмма классов, часть 3

Класс Teacher (рисунгок 5.4) используется для описания данных и функциональности преподавателей в системе школы иностранных языков. Он наследуется от базового класса Account, расширяя его данными о преподавателе и методами для управления курсами и студентами.

Класс Teacher предоставляет широкий функционал для управления курсами, расписанием и взаимодействием с данными о студентах. Его структура и методы позволяют преподавателю эффективно выполнять свои задачи в системе.

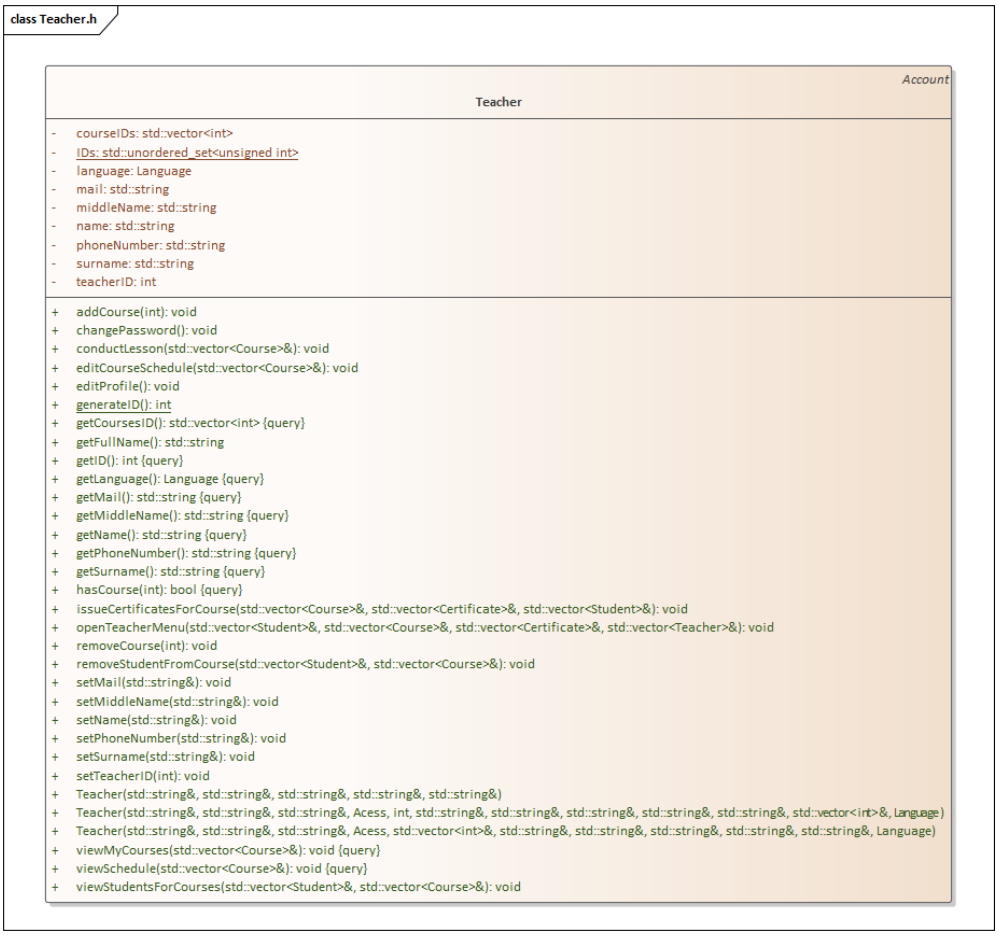


Рисунок 5.4 – Диаграмма классов, часть 4

Классы Account и Admin (рисунок 5.5) реализуют управление учетными записями пользователей системы школы иностранных языков. Они обеспечивают базовую функциональность для работы с учетными данными, а также расширенный функционал для администратора.

Класс Account. Базовый класс для всех учетных записей (администраторы, преподаватели, студенты). Он включает поля и методы для управления учетными данными и доступом пользователя.

Класс Admin расширяет функционал Account и предоставляет методы для управления студентами, преподавателями, курсами и сертификатами. Он реализует интерфейс администратора системы.

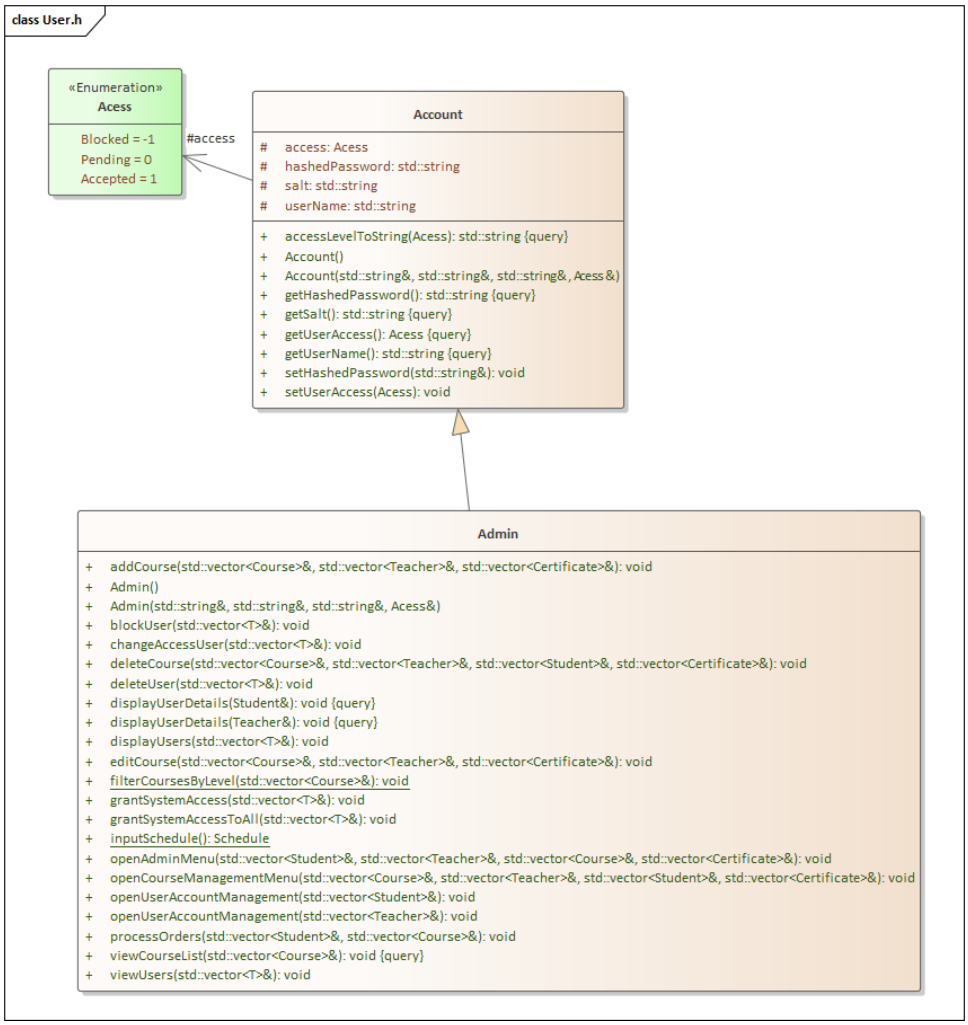


Рисунок 5.5 – Диаграмма классов, часть 5

Классы FileService, UserValidation и ValidationManager (рисунок 5.6) предоставляют вспомогательные функции для управления пользователями, проверки данных и работы с файлами. Они обеспечивают взаимодействие между бизнес-логикой системы и хранилищем данных, а также проверку корректности ввода пользователем.

Класс FileService обеспечивает чтение и запись данных о пользователях, курсах и сертификатах в файлы и из файлов.

Класс UserValidation обеспечивает проверку учетных данных, регистрацию пользователей и обработку авторизации.

Класс ValidationManager обеспечивает проверку корректности ввода данных пользователем (например, телефон, email) и консольный ввод-вывод с цветами.

Эти классы обеспечивают функционал для:

– FileService: работы с файлами (чтение/запись данных);

– UserValidation: проверки учетных данных и управления процессом авторизации/регистрации;

– ValidationManager: проверки пользовательского ввода и удобного консольного интерфейса.

Эти классы составляют основу вспомогательной логики системы.

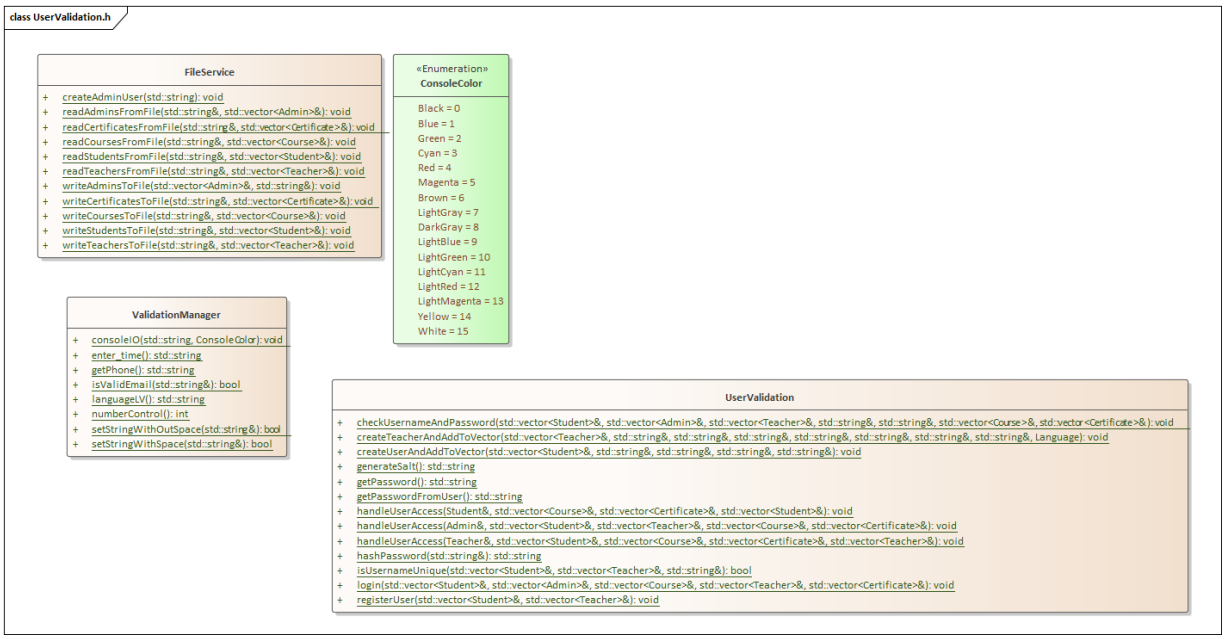


Рисунок 5.6 – Диаграмма классов, часть 6

Классы и методы структурированы таким образом, чтобы максимально эффективно управлять данными школы иностранных языков. Основное внимание уделено гибкости и универсальности решений, что позволяет легко модифицировать или расширять систему.

6 РАЗРАБОТКА И ОПИСАНИЕ ДИАГРАММЫ ВАРИАНТОВ ИСПОЛЬЗОВАНИЯ ПРИЛОЖЕНИЯ

Для наглядного представления всех возможностей разработанного программного продукта используется диаграмма вариантов использования (use case-диаграмма) в нотации UML, представленная на рисунке 6.1.

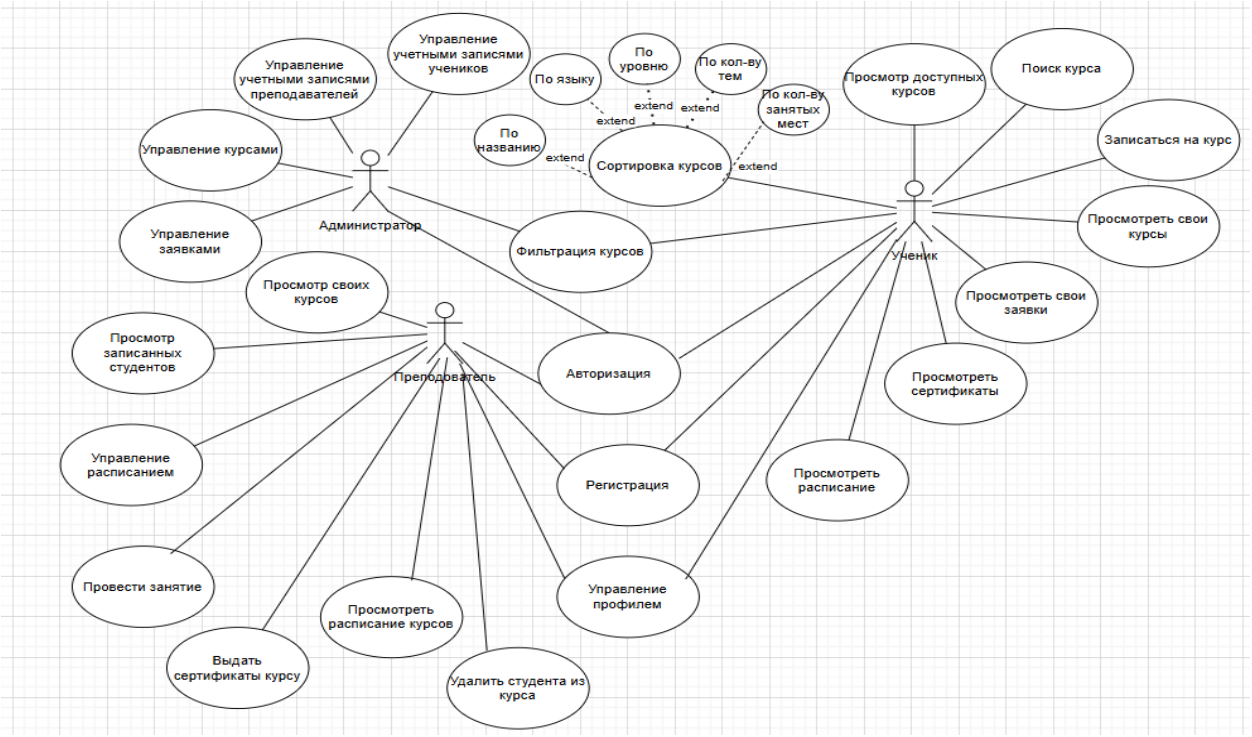


Рисунок 6.1 – UML-диаграмма вариантов использования

Диаграмма вариантов использования для учета деятельности школы иностранных языков демонстрирует основные функции системы и взаимодействие с тремя ролями: администратор, ученик, и преподаватель.

Спецификация функциональных требований подробно отражена в таблице 6.1.

Преподаватель взаимодействует с системой для управления курсами и расписанием, а также для работы с учениками. Ученик использует систему для выбора, оставления заявки и прохождения курсов. Администратор. Администратор отвечает за управление пользователями и курсами.

Таблица 6.1 – Спецификация функциональных требований

Роль	Название операции	Описание
Преподаватель	Регистрация	Создание учетной записи. Регистрация выполнена успешно; необходимо заполнить поля логин и пароль; пользователь с таким логином уже существует.
	Авторизация	Вход выполнен успешно; неправильный логин или пароль
	Управление профилем	Изменить номер телефона; Изменить электронную почту; Изменить пароль; Изменить ФИО. Данные успешно изменены; новый пароль не должен совпадать с со старым.
	Выдать сертификаты курсу	Сертификаты выданы; курс еще не пройден
	Управление расписанием	Изменить дни недели, время, кабинет.
	Просмотр записанных студентов	Список записанных студентов на определенный курс
	Просмотр своих курсов	Список своих курсов с подробной информацией.
	Провести занятие	Занятие успешно проведено; нет доступных курсов
	Удалить студента из курса	Студент удален с курса
	Просмотреть расписание курсов	Возможность просмотреть расписание у своих курсов.

Продолжение таблицы 6.1

Роль	Название операции	Результат
Ученик	Регистрация	Создание учетной записи. Регистрация выполнена успешно; необходимо заполнить поля логин и пароль; пользователь с таким логином уже существует.
	Авторизация	Вход выполнен успешно; неправильный логин или пароль
	Управление профилем	Изменить номер телефона; Изменить электронную почту; Изменить пароль; Изменить ФИО. Данные успешно изменены; новый пароль не должен совпадать с со старым.
	Просмотр расписания	Возможность просмотреть расписание у курсов.
	Записаться на курс	Выбрать подходящий курс, оставить заявку.
	Просмотр своих заявок	Возможность просмотреть оставленные заявки
	Просмотр своих курсов	Возможность просмотреть курсы на которые записан
	Просмотр сертификатов	Возможность просмотреть выданные сертификаты
	Просмотр доступных курсов	Список доступных для записи курсов
Администратор	Авторизация	Вход выполнен успешно; неправильный логин или пароль
	Управление учетными записями учеников	Просмотреть список учеников; изменить права доступа к системе; удаление учетной записи
	Управление учетными записями преподавателей	Просмотреть список преподавателей; изменить права доступа к системе; удаление учетной записи.

Продолжение таблицы 6.1

	Управление курсами	Возможность просмотреть список курсов; добавить курс; редактировать курс; удалить курс; фильтровать курсы по уровню
	Управление заявками	Одобрить или отклонить заявку от учеников на курс

Для школы иностранных языков также важно реализовать функции поиска, сортировки и фильтрации в её системе, что значительно улучшает удобство использования и эффективность взаимодействия с образовательным контентом.

Поиск позволяет студентам быстро находить нужную информацию, без необходимости просматривать весь доступный контент. Это особенно актуально, если количество курсов и ресурсов значительное. Фильтрация помогает ограничить результаты поиска по ключевым параметрам, таким как уровень языка, что ускоряет процесс выбора и позволяет пользователю найти именно то, что ему нужно. Сортировка же помогает упорядочить курсы по различным критериям, что облегчает принятие решения.

Эти функции могут также быть использованы для анализа предпочтений студентов, что поможет школе более точно настроить свои предложения и улучшить маркетинговую стратегию. Понимание того, какие курсы и ресурсы наиболее востребованы, может повлиять на формирование новых программ и повышение качества обучения.

Таким образом, поиск, фильтрация и сортировка – это не просто дополнительные функции, а важные инструменты для улучшения пользовательского опыта, повышения эффективности обучения и поддержки стратегического развития школы иностранных языков.

Спецификация функциональных требований и диаграмма вариантов использования являются ключевыми элементами разработки системы, помогающими определить, какие именно функции должны быть реализованы и как будет происходить взаимодействие с пользователями, что важно для успешной реализации проекта.

7 БЛОК-СХЕМА АЛГОРИТМА РАБОТЫ ВСЕЙ ПРОГРАММЫ И ДВУХ ОСНОВНЫХ МЕТОДОВ

7.1 Схема алгоритма работы всей программы

На рисунке 7.1 приведена блок-схема алгоритма программы. На блок-схеме представлена работа автоматизированной системы учета деятельности школы иностранных языков.

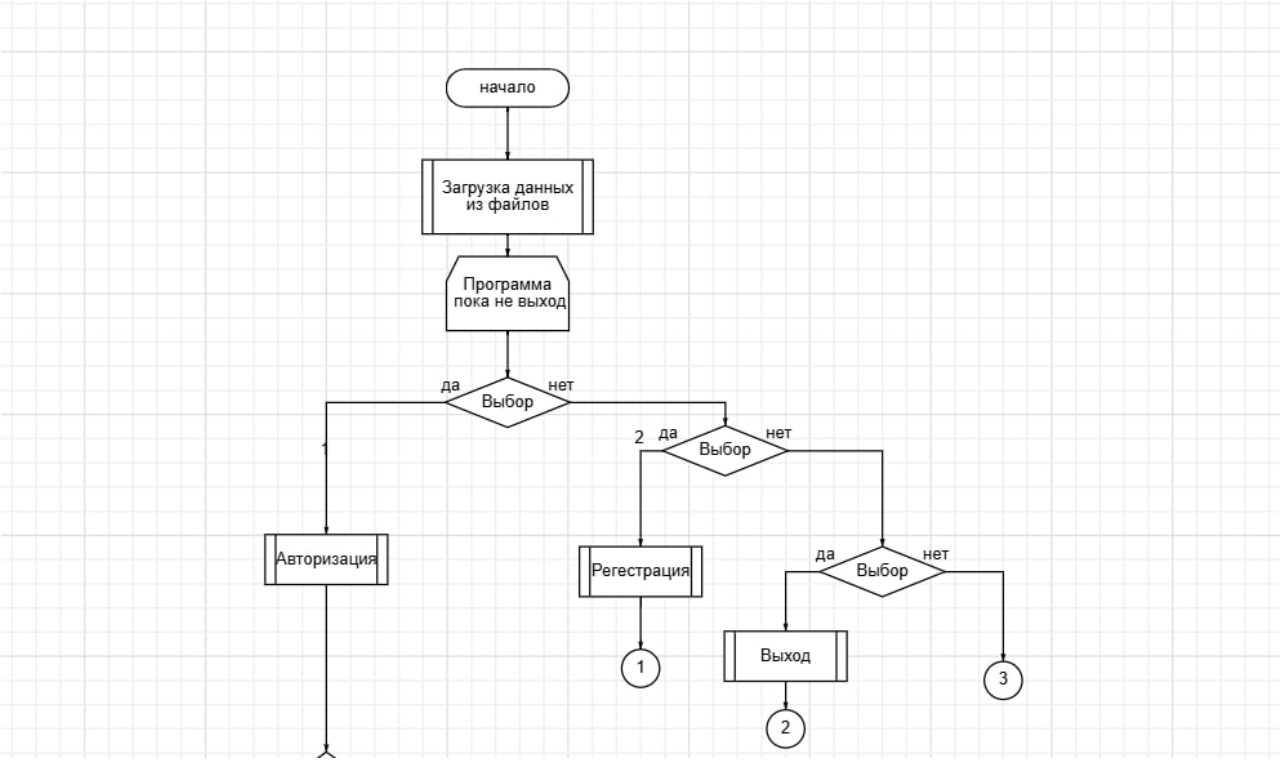


Рисунок 7.1 – Схема алгоритма работы всей программы

Процесс начинается с запуска программы, после чего происходит загрузка необходимых файлов и отображение главного меню. Далее у пользователя появляется выбор: авторизироваться, зарегистрироваться и выйти из программы.

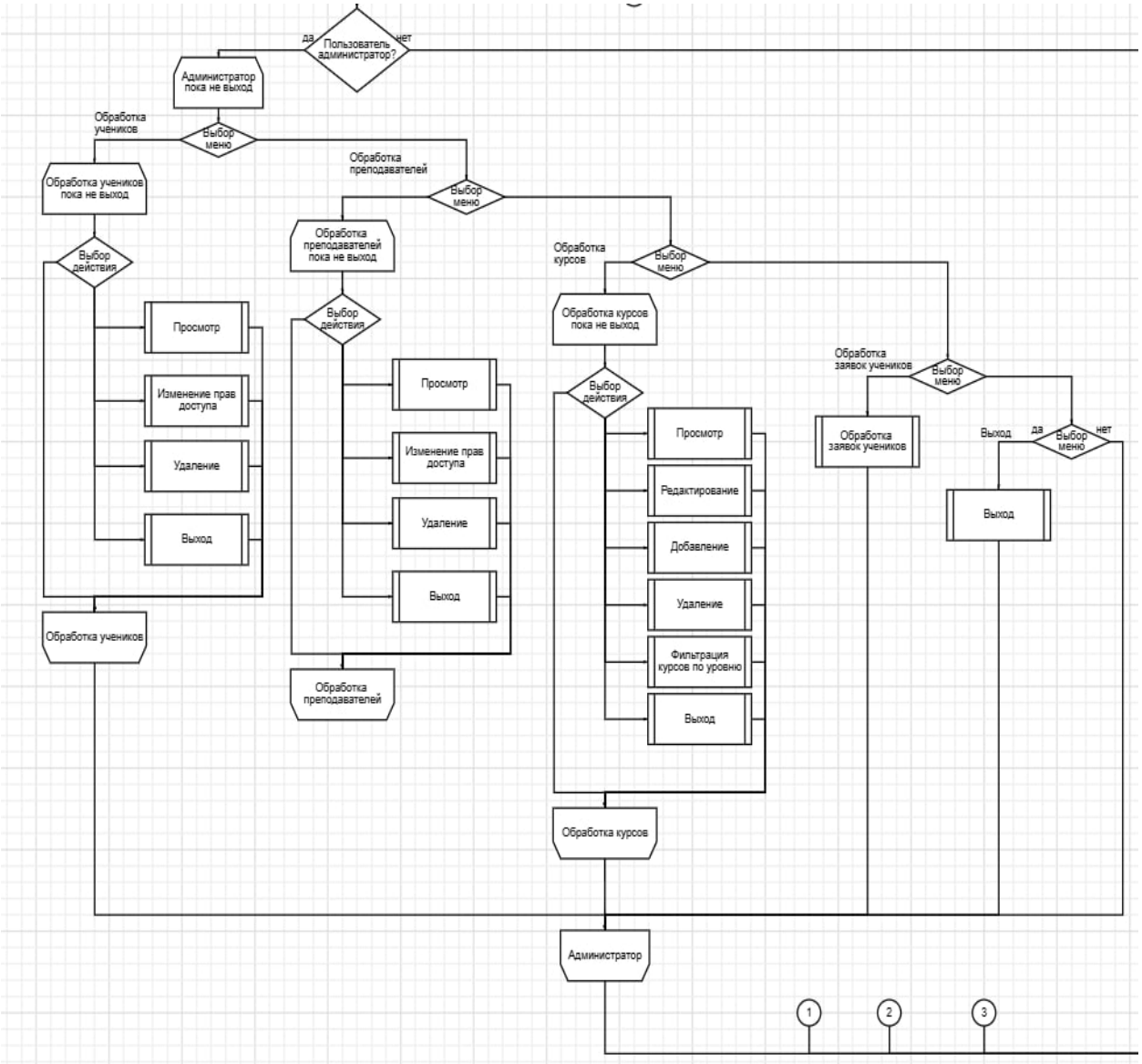


Рисунок 7.2 – Схема алгоритма работы всей программы

После регистрации или авторизации пользователь может зайти под одной из учетных записей: админ, преподаватель, ученик.

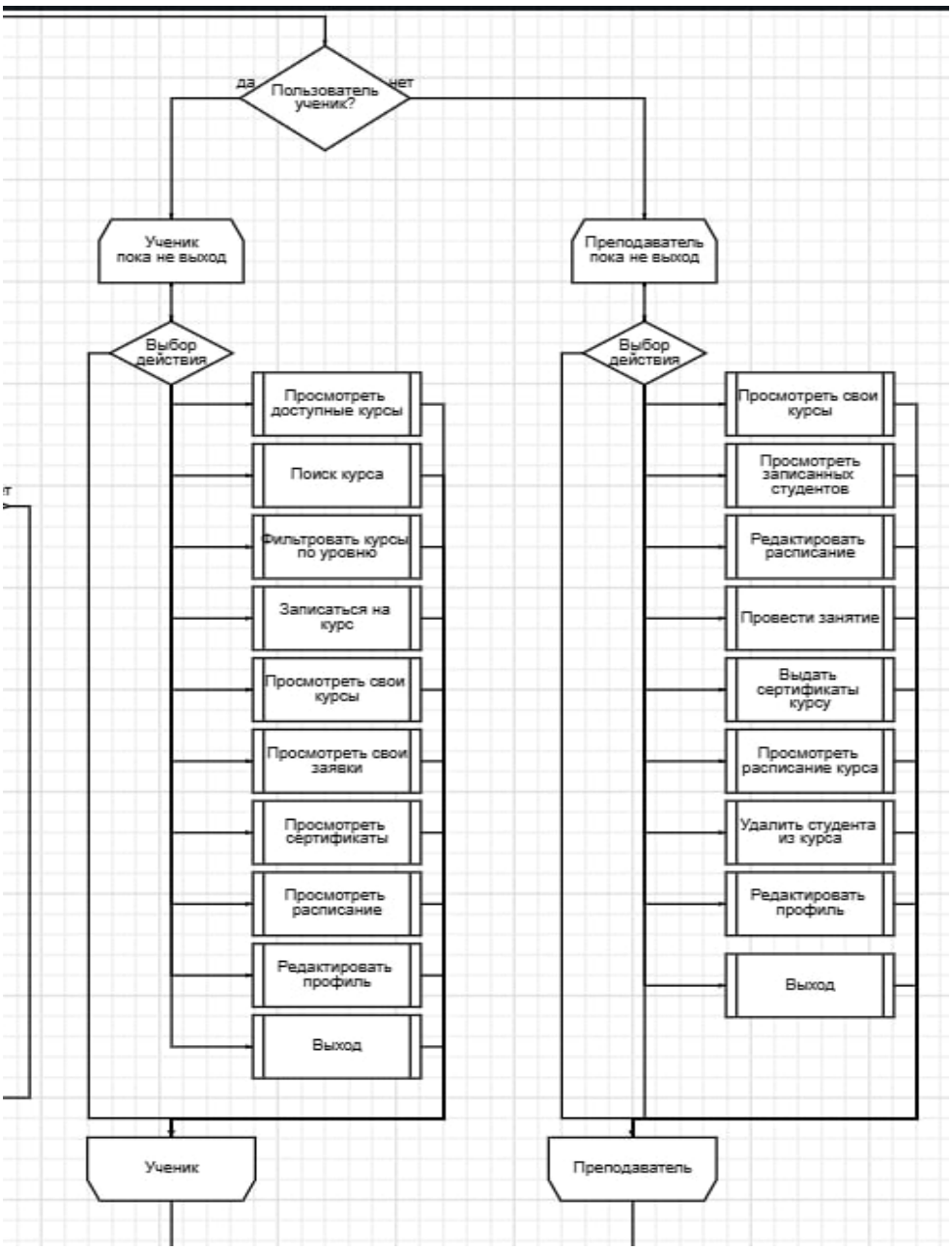


Рисунок 7.3 – Схема алгоритма работы всей программы

После того, как программа по логину и паролю определила роль пользователя, у пользователя появляется соответствующее меню.

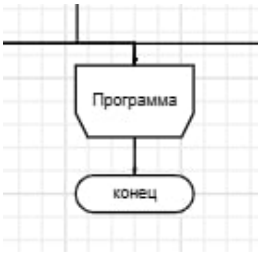


Рисунок 7.4 – Схема алгоритма работы всей программы

Таким образом приведенная блок-схема демонстрирует многоуровневый процесс, что позволяет каждому пользователю эффективно взаимодействовать с системой в рамках своих возможностей.

7.2 Схема алгоритма регистрации пользователя

Процесс регистрации пользователей в автоматизированной системе учета школы иностранных языков. Регистрация представляет собой одну из ключевых функций системы, обеспечивающую создание новых учетных записей для доступа к функционалу программы. Эта операция доступна как для учеников, так и для преподавателей, и является неотъемлемой частью работы системы.

Схема алгоритма регистрации пользователя представлена на рисунке 7.5.

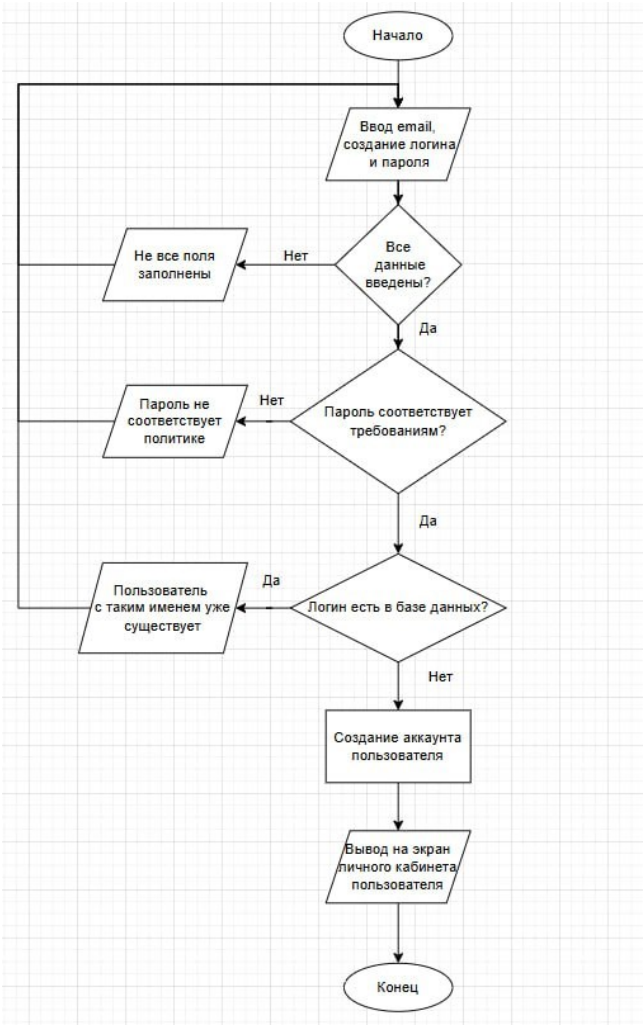


Рисунок 7.5 – Схема алгоритма регистрации пользователя

Процесс регистрации пользователя начинается с ввода выбранного пользователем логина и пароля. Если введены все данные, то идет проверка на ввод пароля, который должен соответствовать требованиям. В противном случае выводится ошибка, что не все поля заполнены.

После проверки пароля на соответствие требованиям, проверяется логин на уникальность. Если логин уникален, то создается аккаунт пользователя и выводится на экран его личный кабинет. В противном случае, выводит сообщение, что пользователь с таким именем уже существует, необходимо заново ввести логин и пароль.

Процесс регистрации обеспечивает удобство. Пошаговый ввод данных с подробными сообщениями об ошибках делает регистрацию понятной и доступной. Безопасность: проверка уникальности данных, таких как логин, предотвращает дублирование учетных записей.

7.3 Схема алгоритма авторизации пользователя

Схема алгоритма авторизации пользователя представлена на рисунке 7.6.

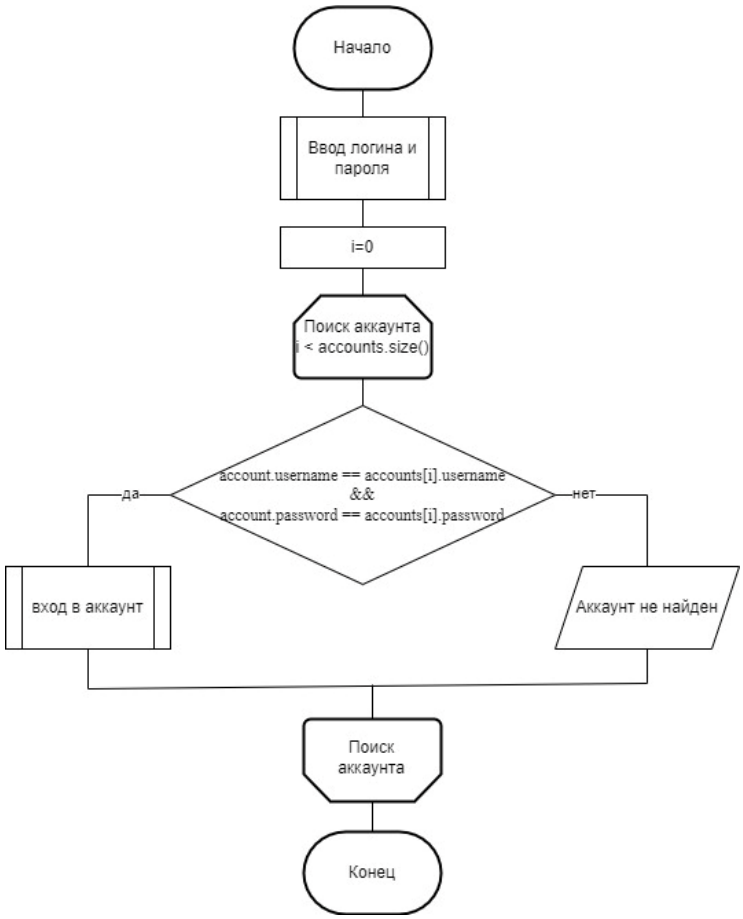


Рисунок 7.6 – Схема алгоритма авторизации пользователя

Процесс авторизации пользователя в системе учета школы иностранных языков. Авторизация является важным этапом, который обеспечивает доступ зарегистрированных пользователей к функционалу программы в зависимости от их роли (ученик, преподаватель или администратор). Авторизация пользователя начинается с ввода логина и пароля, которые сверяются с данными, хранящимися в файле (базе). Если проверка успешна, система предоставляет доступ к интерфейсу в зависимости от роли пользователя. В случае ошибок, таких как неверный пароль или логин, программа отображает соответствующее сообщение и предлагает повторить попытку.

Процесс авторизации играет ключевую роль в обеспечении безопасности данных. Проверка логина и пароля предотвращает несанкционированный доступ. Разграничения доступа: система предоставляет доступ только к функционалу, соответствующему роли пользователя. Удобства использования: встроенные уведомления об ошибках помогают пользователям быстро исправлять проблемы, такие как неверный ввод данных.

Таким образом, авторизация обеспечивает надежный и структурированный доступ пользователей к функционалу системы, что является неотъемлемой частью работы школы иностранных языков [7].

8 ОПИСАНИЕ АЛГОРИТМА ЗАПУСКА ПРИЛОЖЕНИЯ, ЕГО ИСПОЛЬЗОВАНИЯ, РЕЗУЛЬТАТЫ РАБОТЫ, ТЕСТИРОВАНИЯ ОБРАБОТКИ ОШИБОК

8.1 Алгоритм запуска приложения

Для того чтобы автоматизированная система учета деятельности школы иностранных языков могла работать на другом устройстве, это устройство должно иметь операционную систему Windows 7 и выше, а также Microsoft .NET Framework.

После чего нужно скачать файл «Учет курсов» с разрешением .exe, а также для полной работы приложения необходимы текстовые файлы: «admins», «certificate», «courses», «students» и «teachers» с разрешением .txt . Далее необходимо все эти файл добавить в одну папку.

Запускаем приложение, два раза нажав на файл с разрешением .exe.

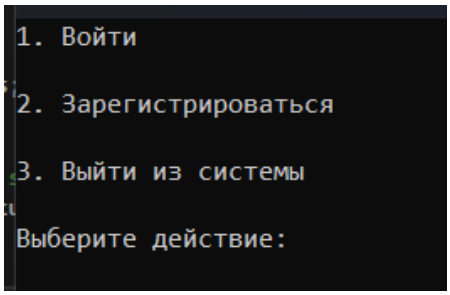


Рисунок 8.1 – Скриншот главного окна приложения после запуска

8.2 Руководство пользователя

При запуске программы пользователю предоставляется выбор регистрация если пользователь не авторизирован, а также авторизации, если он уже авторизирован. Если пользователь уже авторизирован он имеет возможность войти в систему в Администратора. Меню представлено на рисунке 8.1.

Если же пользователь не авторизирован он может выбрать пункт «Зарегистрироваться», после чего он может зарегистрироваться в системе как преподаватель или как ученик. Меню регистрации представлено на рисунке 8.2.



Рисунок 8.2 – Меню регистрации

Регистрация происходит следующим образом: пользователь выбирает роль (преподаватель или ученик), далее вводит логин и пароль, если введенный пользователем логин уже имеется, то программа запросит повторный ввод логина и пароля, после чего, если логин отсутствует, регистрация пройдет успешно, и пользователь увидит соответствующее сообщение. Регистрация пользователя в качестве преподавателя приведена на рисунке 8.3.

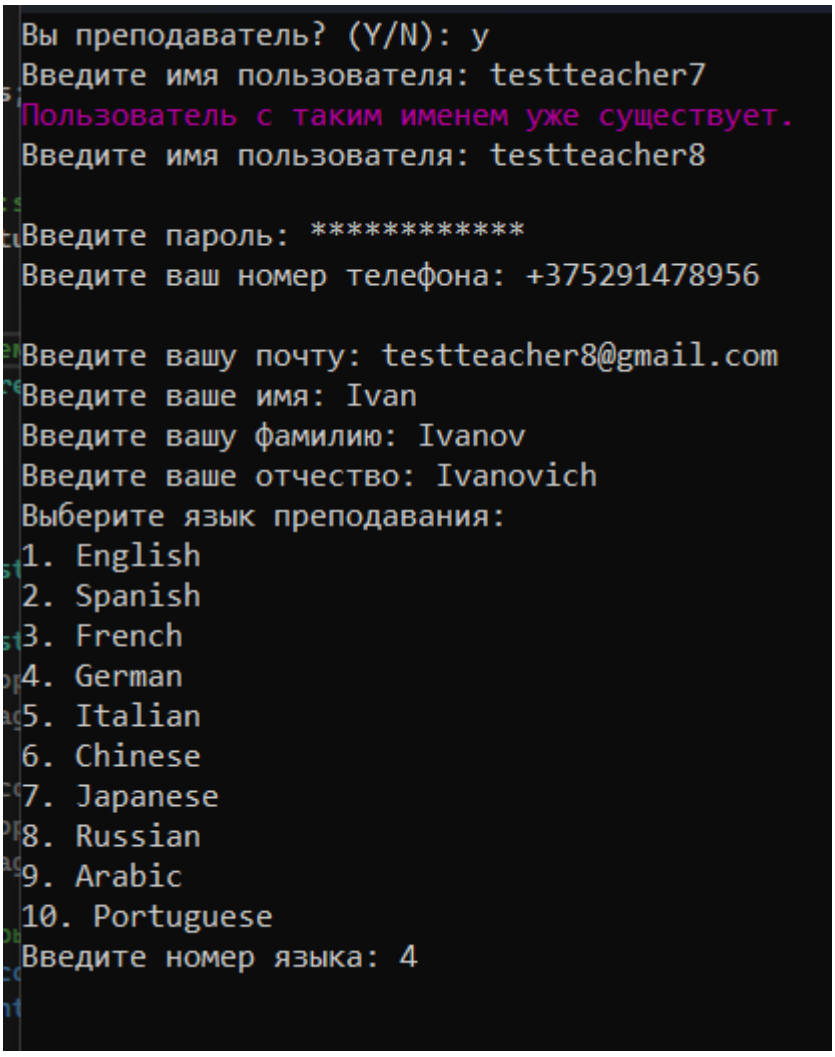


Рисунок 8.3 – Регистрация пользователя в качестве преподавателя

Регистрация пользователя в качестве ученика приведена на рисунке 8.4.

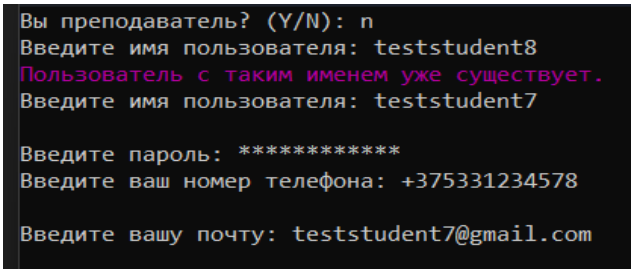


Рисунок 8.4 – Регистрация пользователя в качестве ученика

Перед тем, как войти в систему после регистрации преподавателю или ученику требуется подтверждение администратора. Это представлено на рисунке 8.5.



Рисунок 8.5 – Ожидание подтверждения от администратора

Далее рассмотрим какие функции доступны пользователю при входе в систему в качестве администратора. На рисунке 8.6 представлен вход.

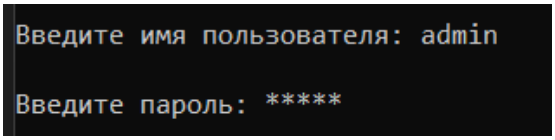


Рисунок 8.6 – Ввод логина и пароля

После того как администратор вошел в систему введя логин и пароль, ему отрывается меню администратора (рисунок 8.7).

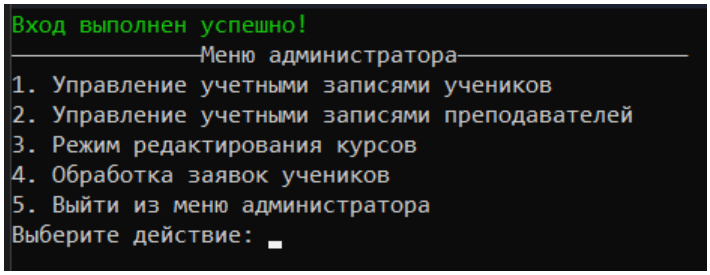


Рисунок 8.7 – Меню администратора

После чего администратор выбирает из один из пунктов.

Выбрав пункт «Управление учетными записями учеников» открывается меню работы с учетными записями учеников (рисунок 8.8). В данном меню есть возможности просмотреть список учеников, изменить права доступа к системе и удаление учетной записи.

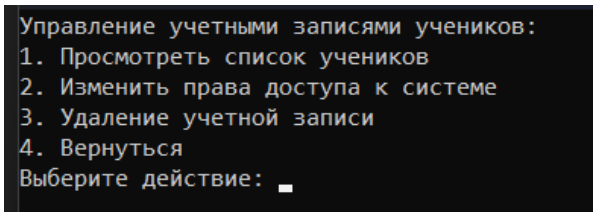


Рисунок 8.8 – Меню работы с учетными записями учеников

При выборе пункта «Просмотреть список учеников» выводится таблица с учениками (рисунок 8.9).

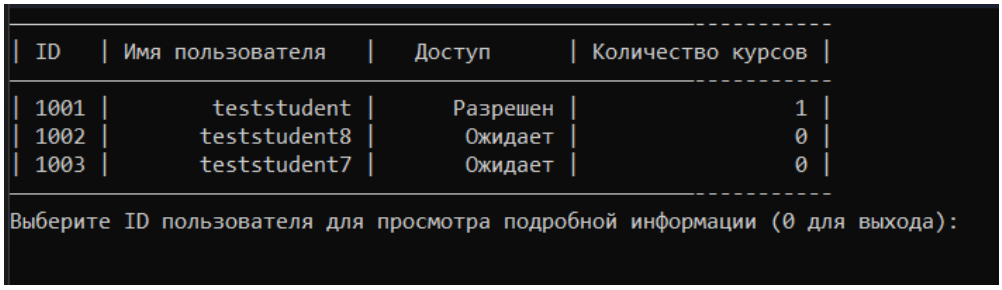


Рисунок 8.9 – Таблица учеников

Также предоставляется возможность просмотреть более подробную информацию о любом из учеников. Для этого ученика выбирают по ID (рисунок 8.10).

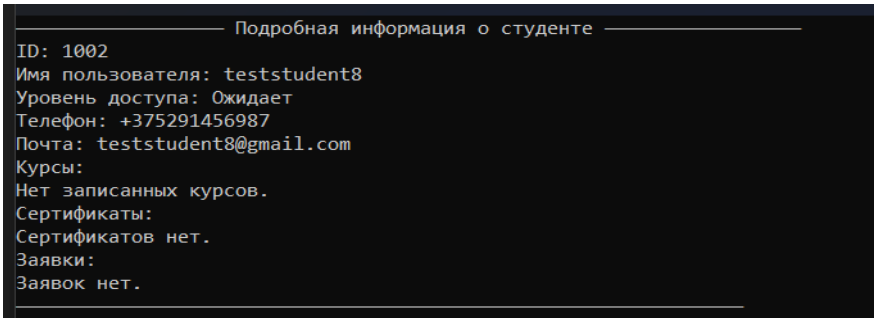


Рисунок 8.10 – Подробная информация о студенте

Если же выбрать пункт «Изменить права доступа к системе» то откроется выбор, при котором можно заблокировать или разблокировать пользователя, а также разрешить доступ к системе всем новым пользователям (рисунок 8.11).

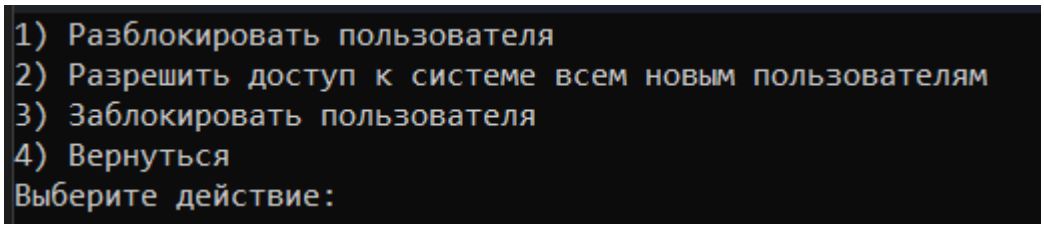


Рисунок 8.11 – Меню права доступа

При выборе пункта «Удаление учетной записи», программа запросит ввести имя пользователя, которого необходимо удалить (рисунок 8.12).

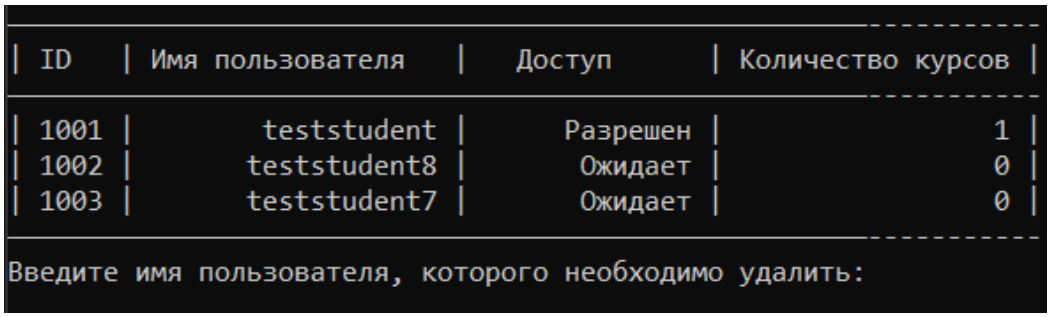


Рисунок 8.12 – Удаление учетной записи ученика

Рассмотрим еще одну опцию, которая доступна администратору – это управление учетными записями преподавателей (рисунок 8.13). Выбрав этот пункт открывается меню работы с учетными записями преподавателей. В данном меню есть возможности просмотреть список преподавателей, изменить права доступа к системе и удаление учетной записи.

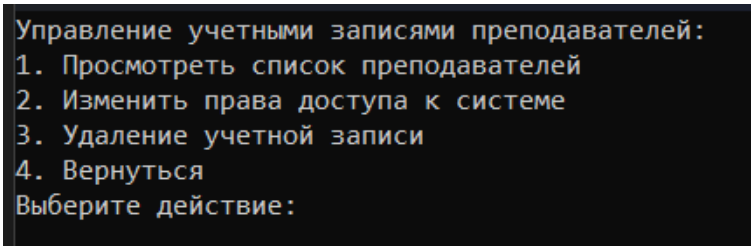


Рисунок 8.13 – Меню работы с учетными записями преподавателей

При выборе пункта «Просмотреть список преподавателей» выводится таблица с преподавателями (рисунок 8.14).

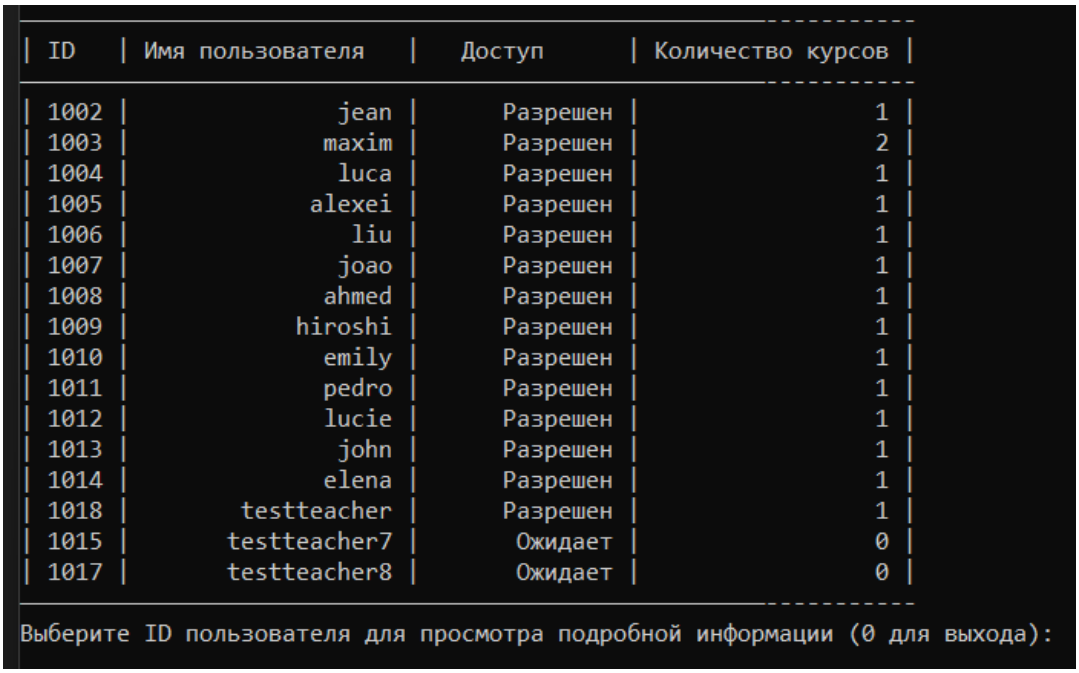


Рисунок 8.14 – Таблица преподавателей

Также предоставляется возможность просмотреть более подробную информацию о любом из преподавателей. Для этого преподавателя выбирают по ID (рисунок 8.15).

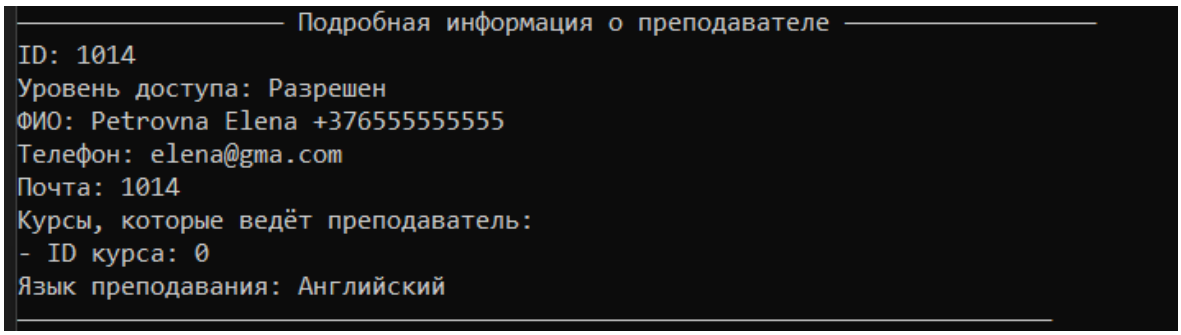


Рисунок 8.15 – Подробная информация о преподавателе

Если же выбрать пункт «Изменить права доступа к системе» то откроется выбор, при котором можно заблокировать или разблокировать пользователя, а также разрешить доступ к системе всем новым пользователям (рисунок 8.16).

```
1) Разблокировать пользователя
2) Разрешить доступ к системе всем новым пользователям
3) Заблокировать пользователя
4) Вернуться
Выберите действие:
```

Рисунок 8.16 – Меню права доступа

При выборе пункта «Удаление учетной записи», программа запросит ввести имя пользователя, которого необходимо удалить (рисунок 8.17).

```
| ID   | Имя пользователя   |   Доступ      | Количество курсов |
| 1002 |               jean |      Разрешен |                 1 |
| 1003 |              maxim |      Разрешен |                 2 |
| 1004 |               luca |      Разрешен |                 1 |
| 1005 |             alexei |      Разрешен |                 1 |
| 1006 |                liu |      Разрешен |                 1 |
| 1007 |               joao |      Разрешен |                 1 |
| 1008 |              ahmed |      Разрешен |                 1 |
| 1009 |            hiroshi |      Разрешен |                 1 |
| 1010 |              emily |      Разрешен |                 1 |
| 1011 |              pedro |      Разрешен |                 1 |
| 1012 |              lucie |      Разрешен |                 1 |
| 1013 |               john |      Разрешен |                 1 |
| 1014 |              elena |      Разрешен |                 1 |
| 1018 |        testteacher |      Разрешен |                 1 |
| 1015 |       testteacher7 |       Ожидает |                 0 |
| 1017 |       testteacher8 |       Ожидает |                 0 |
Введите имя пользователя, которого необходимо удалить:
```

Рисунок 8.17 – Удаление учетной записи преподавателя

Если же пользователь выбрал пункт Режим редактирования курсов, то ему откроется меню Управление курсами (рисунок 8.18).

```
Управление курсами
1. Просмотреть список курсов
2. Добавить курс
3. Редактировать курс
4. Удалить курс
5. Фильтровать курсы по уровню
6. Вернуться в главное меню администратора
Выберите действие:
```

Рисунок 8.18 – Управление курсами

При выборе пункта «Просмотреть список курсов» пользователю откроется список курсов (рисунок 8.19).

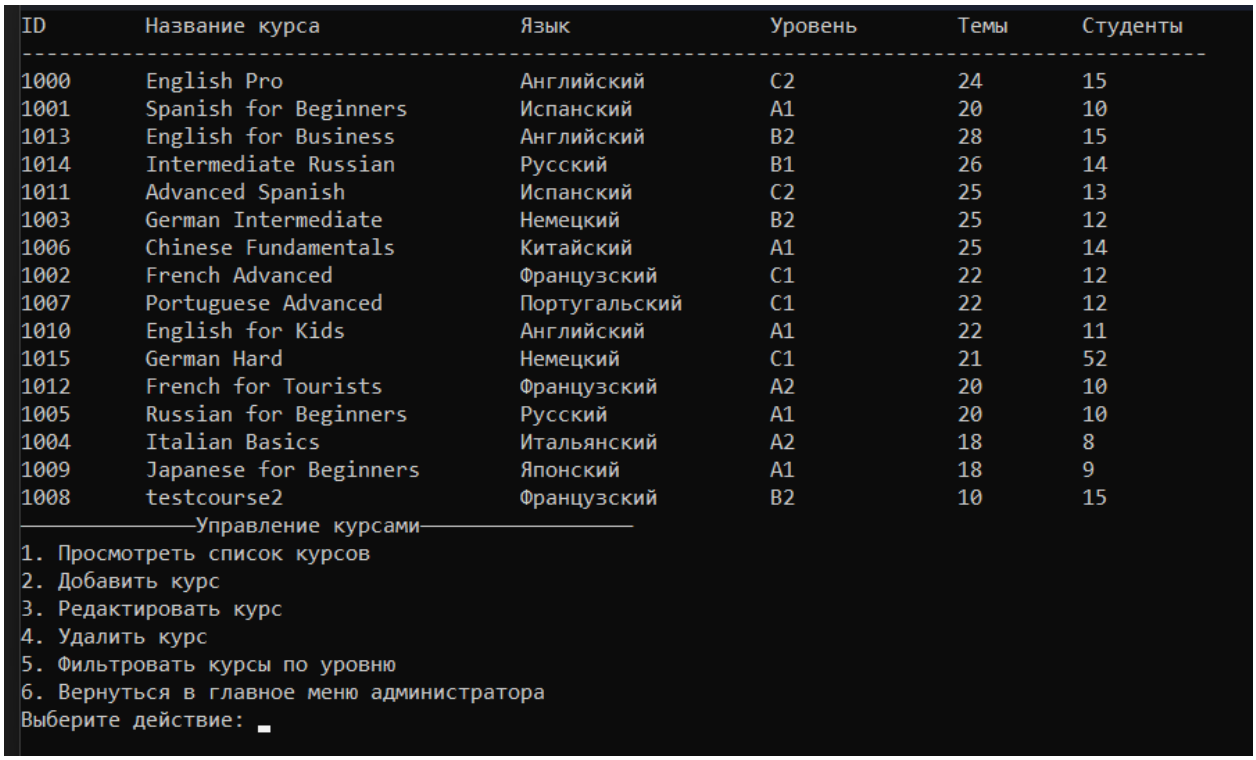


Рисунок 8.19 – Список курсов

При выборе пункта «Добавить курс» пользователь сможет добавить новый курс. Сначала он введет название курса, затем выберет язык, после чего появится список преподавателей, и пользователь выберет одного из них. Далее будет введен уровень языка и количество тем, а также максимальное количество учеников в группе. После чего, пользователю необходимо будет выбрать дату и время занятий. Если пользователь введет время окончания занятия, которое будет раньше начала занятия, то программа выведет сообщение об ошибке. Добавление курса представлено на рисунке 8.20.

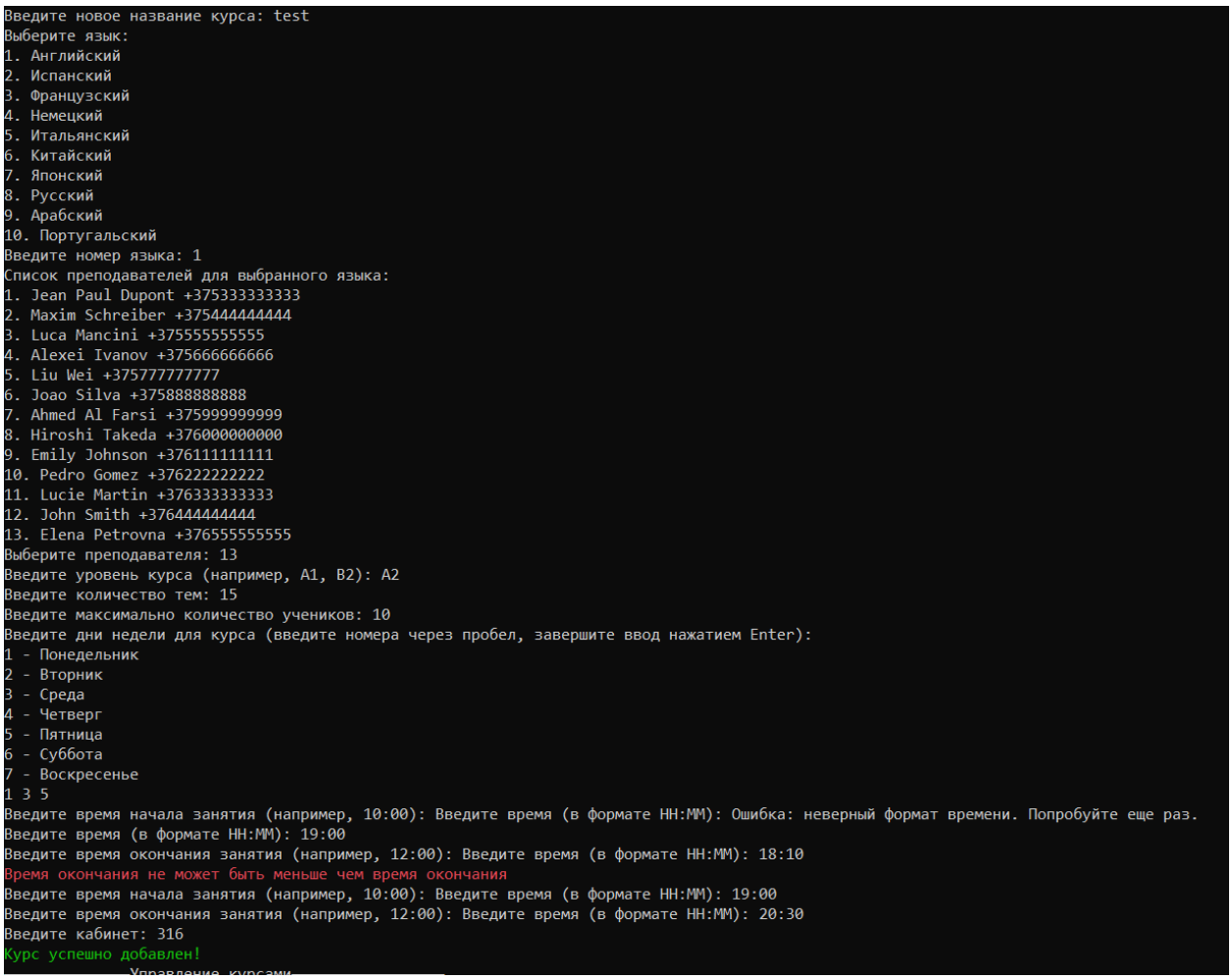


Рисунок 8.20 – Добавление курса

Удаление курса возможно по ID. Рисунок 8.21.

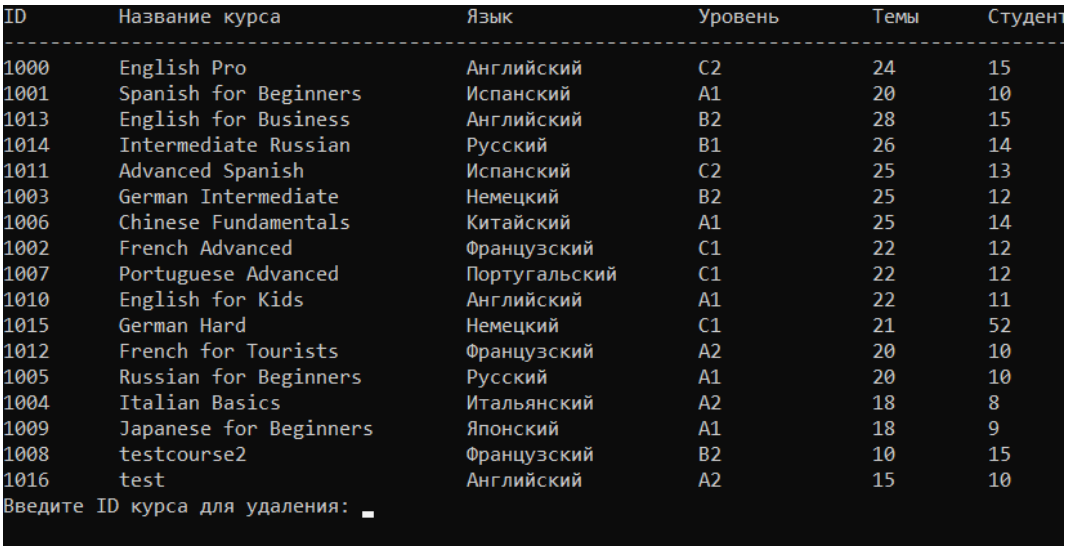


Рисунок 8.21 – Удаление курса

Фильтрация курсов доступна по уровню языка (рисунок 8.22)

```
Введите минимальный уровень (например, A1): A2
Введите максимальный уровень (например, B2): B1

Курсы с уровнями от A2 до B1:
┌──────────┬──────────────────────┬──────────────┬──────────┬──────────┐
| ID курса |       Название       |     Язык     |  Уровень | Кол-во тем |
├──────────┼──────────────────────┼──────────────┼──────────┼──────────┤
| 1014     | Intermediate Russian | Русский      | B1       | 26       |
| 1012     | French for Tourists  | Французский  | A2       | 20       |
| 1004     | Italian Basics       | Итальянский  | A2       | 18       |
| 1014     | test                 | Английский   | A2       | 15       |
└──────────┴──────────────────────┴──────────────┴──────────┴──────────┘
            Управление курсами
```

Рисунок 8.22 – Фильтрация курса

Далее рассмотрим какие функции доступны пользователю при входе в систему в качестве ученика. На рисунке 8.23 представлен вход.

```
Вход выполнен успешно!
Добро пожадовать teststudent8
Ваша почта: teststudent8@gmail.com
Ваш номер телефона: +375291456987
──────────────Меню студента──────────────
1. Просмотреть доступные курсы
2. Поиск курса
3. Фильтровать курсы по уровню
4. Записаться на курс
5. Просмотреть свои курсы
6. Просмотреть свои заявки
7. Просмотреть сертификаты
8. Просмотреть расписание
9. Редактировать профиль
0. Выйти из меню
Выберите действие:
```

Рисунок 8.23 – Выполнен вход в аккаунт ученика

Выбрав пункт «Просмотреть доступные курсы», пользователи могут их также отсортировать по названию, по языку, по уровню языка, по количеству тем, по количеству свободных мест (рисунок 8.24)

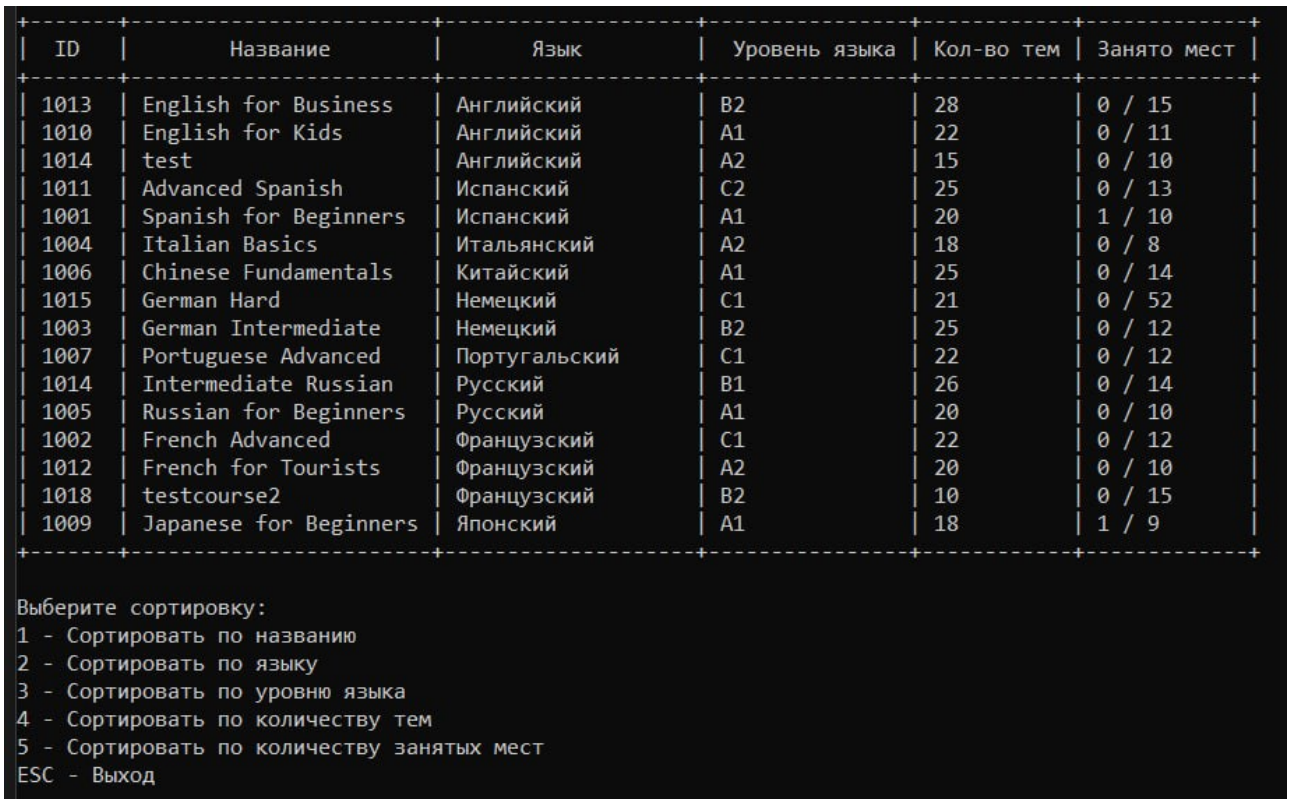


Рисунок 8.24 – Просмотр доступных курсов

Поиск курса осуществляется постепенно, по вводимым пользователем символам. Таблица поиска обновляется с каждым новым введенным символом (рисунок 8.25).

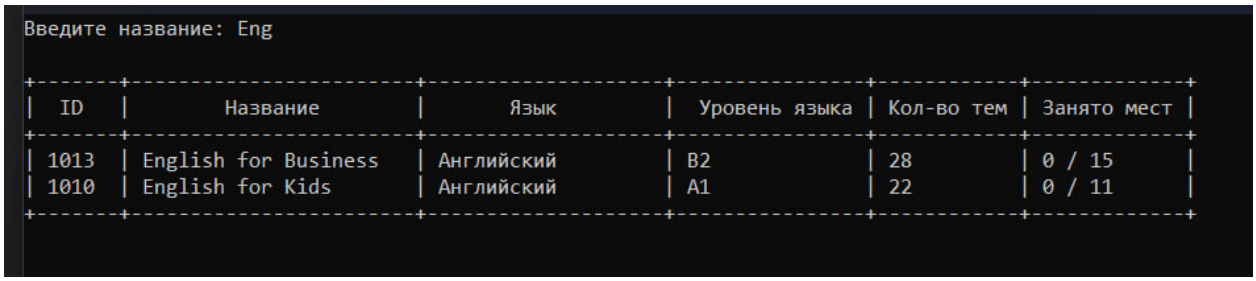


Рисунок 8.25 – Поиск курсов

Фильтрация курсов по уровню представлена на рисунке 8.26.

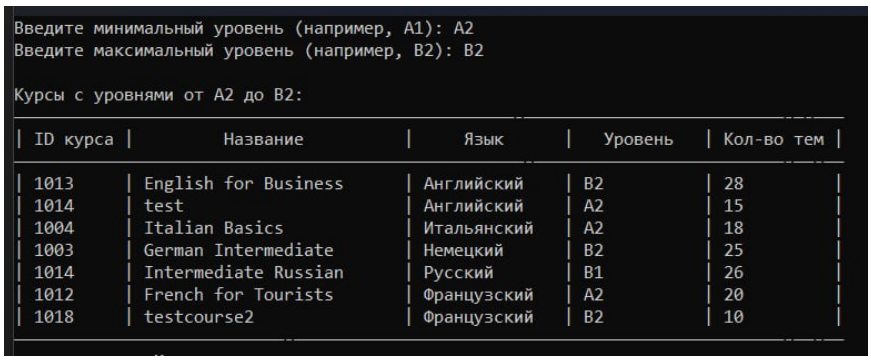


Рисунок 8.26 – Фильтрация курсов

Выбрать курс для подачи заявки можно по ID. (рисунок 8.27.1).

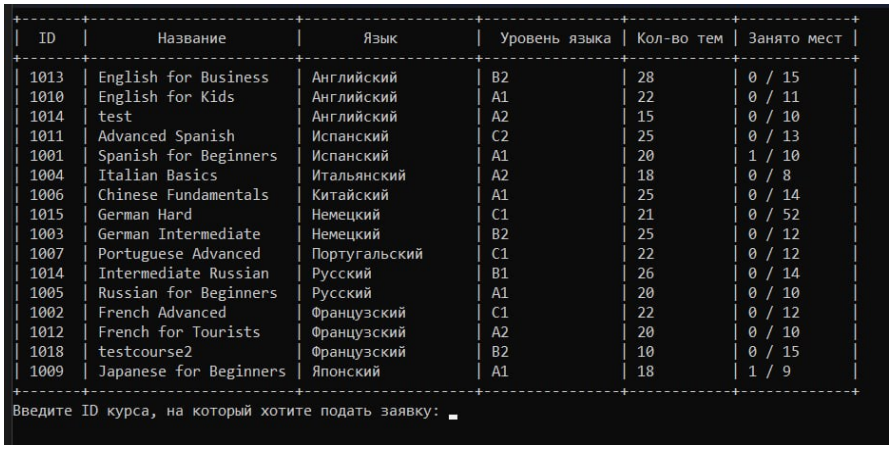


Рисунок 8.27.1 – Выбор курса для подачи заявки

Далее эту заявку одобряет или не одобряет администратор (рисунок 8.27.2).

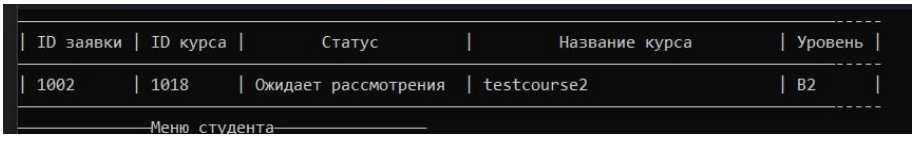


Рисунок 8.27.2 – Заявка ожидает рассмотрения

Администратор одобрил заявку (рисунок 8.27.3).

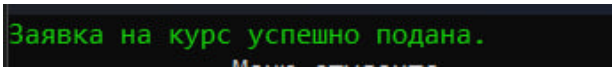


Рисунок 8.27.3 – Заявка принята

Пользователь также может просмотреть свои курсы, заявки, сертификаты и расписание занятий.

Пользователь может редактировать свой профиль, а именно может редактировать номер телефона, электронную почту, пароль (рисунок 8.28).

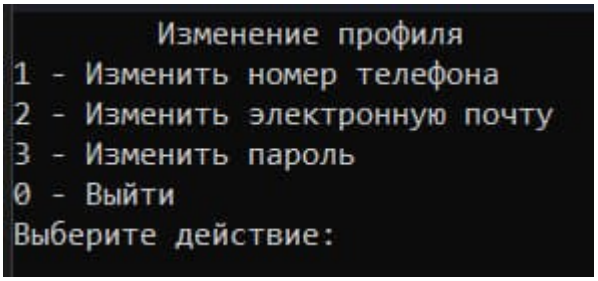


Рисунок 8.28 – Изменение профиля

Далее рассмотрим какие функции доступны пользователю при входе в систему в качестве преподавателя. На рисунке 8.29 представлен вход.

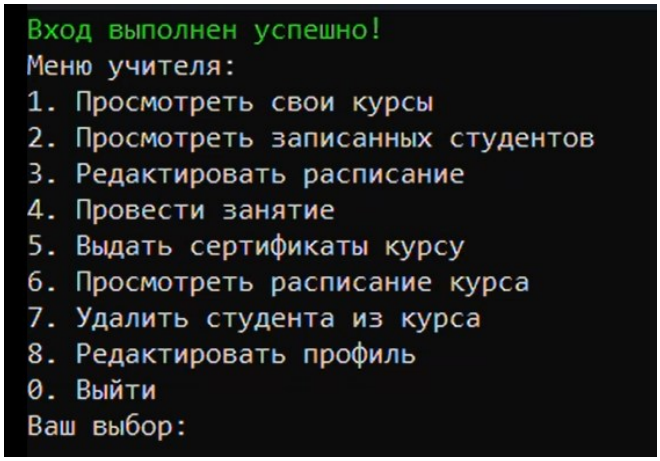
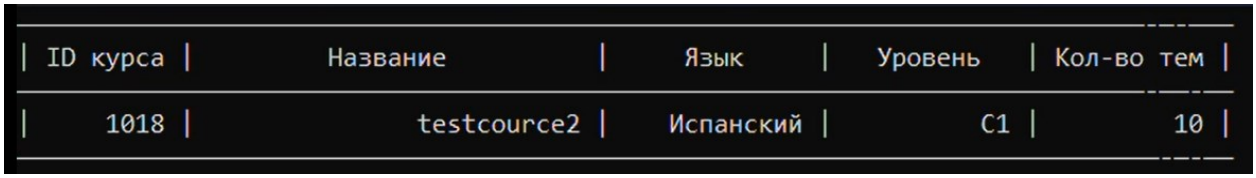


Рисунок 8.29 – Выполнен вход в аккаунт преподавателя

Выбрав пункт «Просмотреть свои курсы» преподаватель увидит курсы, которые он преподает (рисунок 8.30).



ID курса	Название	Язык	Уровень	Кол-во тем
1018	testcource2	Испанский	C1	10

Рисунок 8.30 – Список курсов

Выбрав пункт «Посмотреть записанных студентов» у преподавателя попросят ввести ID того курса, список учеников которого ему интересен (рисунок 8.31)

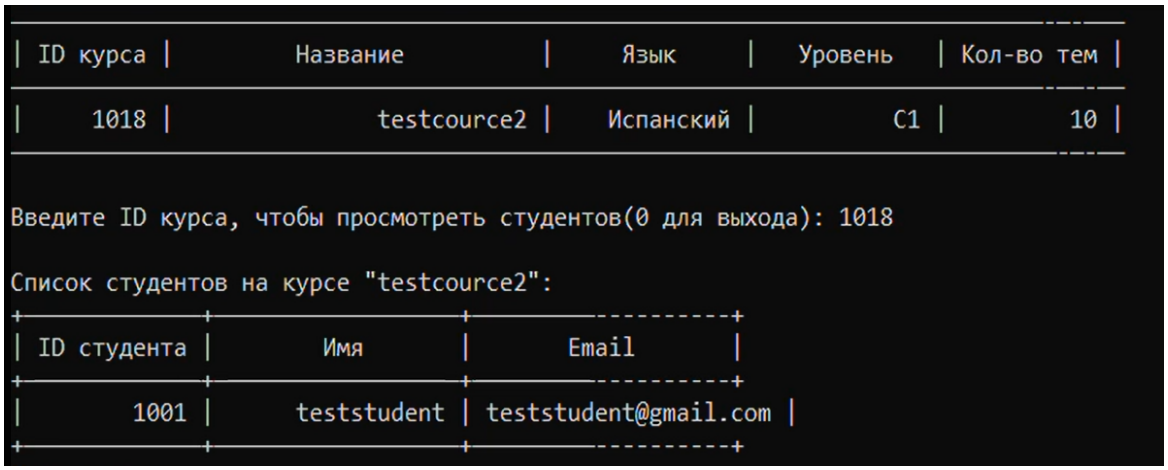


Рисунок 8.31 – Записанные ученики

При выборе пункта «Просмотреть расписание курса» преподаватель увидит дни недели, время и кабинет в котором проводят занятия интересуемого курса. Расписание также можно редактировать, для этого необходимо выбрать пункт «Редактировать расписание». Редактирование расписания представлено на рисунке 8.32.

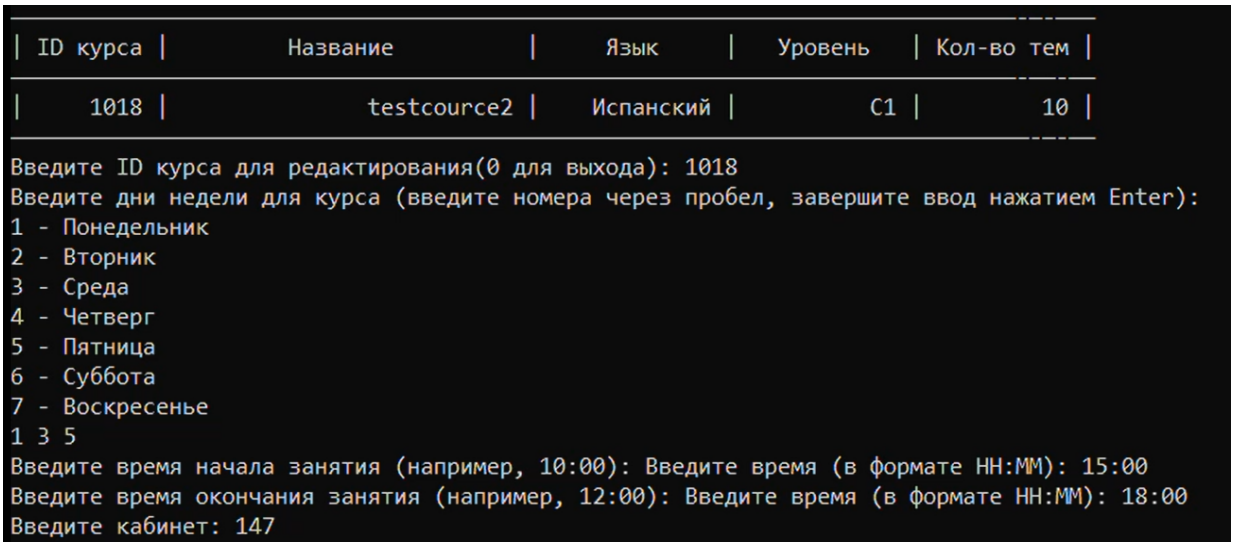


Рисунок 8.32 – Редактирование расписания

Также в функционале у преподавателя есть возможность провести занятие. Для этого из всех курсов он выбирает нужный по ID и выбирает пункт «провести занятие». Только после того, как все занятия этого курса пройдут, преподаватель сможет выдать сертификат студентам.

Преподаватель может удалять студентов. Для этого он сначала выбирает необходимый курс, потом необходимого студента (рисунок 8.33).

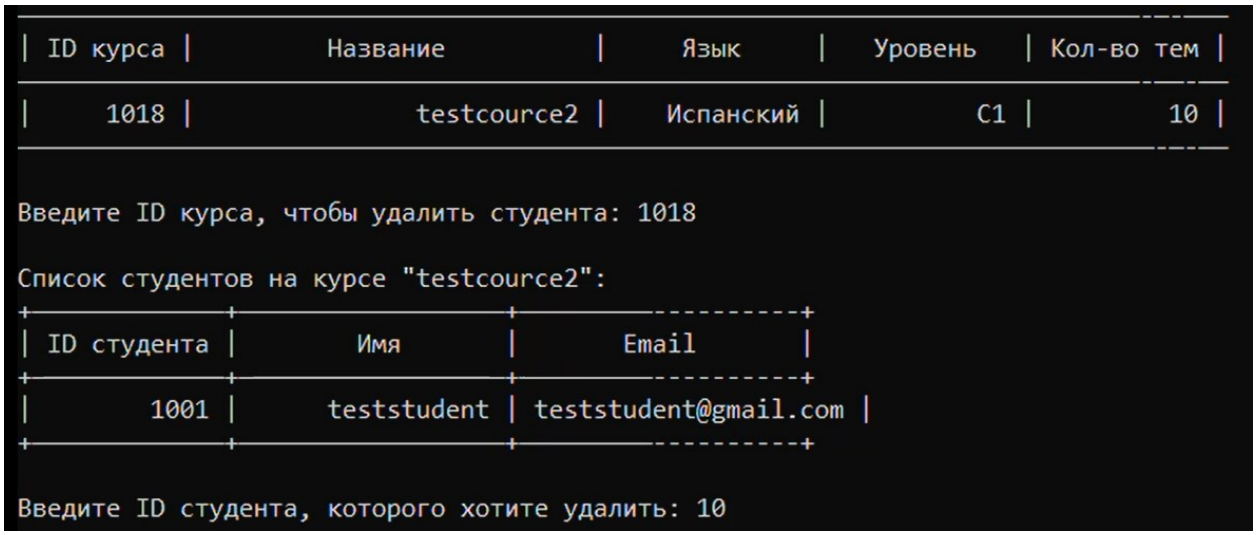


Рисунок 8.33 – Удаление студента

Также преподаватель может выбрать «Редактировать профиль», чтобы изменить такие данные как номер телефона, электронную почту, пароль и ФИО (рисунок 8.34).

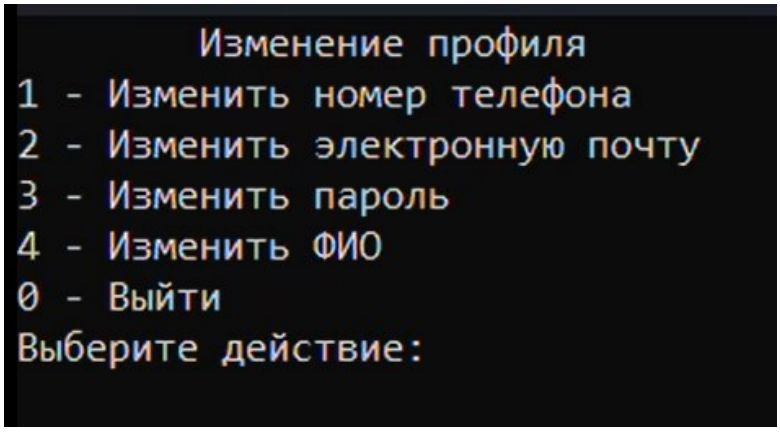


Рисунок 8.34 – Редактировать профиль

Таким образом мы описали руководство пользователя. Если у администратора, студента или преподавателя возникнут вопросы по программе, они смогут обратиться к данному руководству и без труда разобраться в возникшем вопросе.

8.3 Тестирование работы приложения

В данном разделе рассмотрено тестирование и проверка работоспособности программы учета деятельности школы иностранных языков. Основное внимание будет уделено обнаружению и обработке исключительных ситуаций, которые могут возникнуть в процессе использования программы. Особое внимание будет уделено проверке ввода данных пользователем через консоль, так как это один из основных способов взаимодействия пользователя с программой.

При входе в программу всплывает меню с выбором действий: войти в учётную запись, если такая имеется, зарегистрироваться или выйти из программы. Рисунок 8.35. При некорректном вводе выводиться сообщение о ошибки и просит ввести снова. Рисунок 8.36.

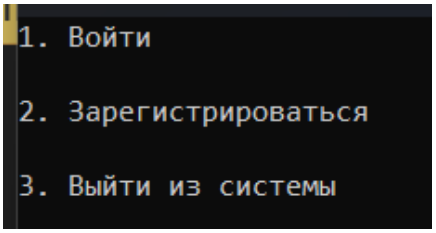


Рисунок 8.35 – Меню входа в программу

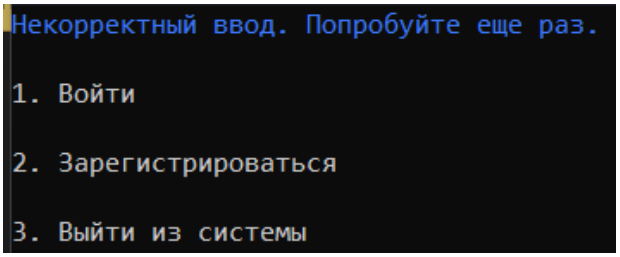


Рисунок 8.36 – Ошибка ввода

После выбора действия «Вход» программа просит ввести логин и пароль. В случае, если такой учётной записи в программе нет, либо логин или пароль были введены неправильно, выведется следующее сообщение. Рисунок 8.37.

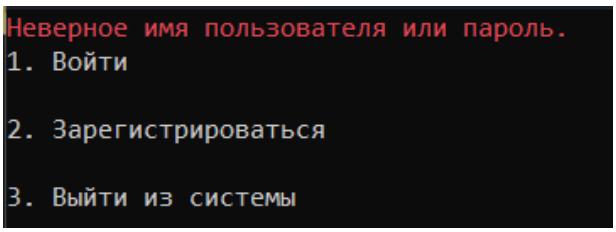


Рисунок 8.37 – Сообщение о неправильном вводе логина или пароля

Если был выбран пункт «Зарегистрироваться», программа просит ввести логин и пароль. Если же учётная запись с введенным логином существуют, то программа выводит сообщение о ошибке, рисунок 8.38002E

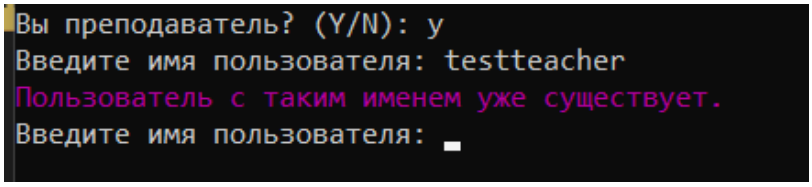


Рисунок 8.38 – Сообщение об ошибке

Если пользователь решит выбрать несуществующий пункт меню программа выведет сообщение об ошибке (рисунок 8.39), также если пользователь попробует ввести символ или букву, то ничего не произойдет так как программа попросту не даст этого сделать.

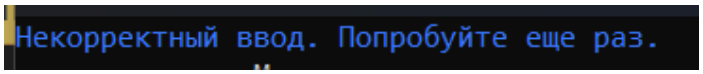


Рисунок 8.39 – Сообщение об ошибке

При вводе любых текстовых или числовых данных в неправильное место, приложение не дает вводить пользователю иных символов кроме английских букв и арабских цифр.

В программе реализована проверка на ввод электронной почты (рисунок 8.40, 8.41).

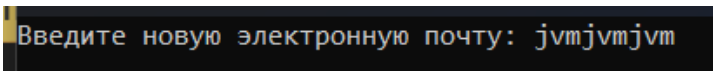


Рисунок 8.40 – Сообщение о некорректном формате ввода

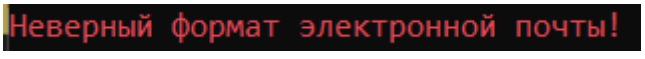


Рисунок 8.41 – Сообщение о некорректном формате ввода

При вводе номера телефона программе не даст ввести больше количество цифр, чем 9, так как из такого количества цифр состоит телефон (рисунок 8.42).

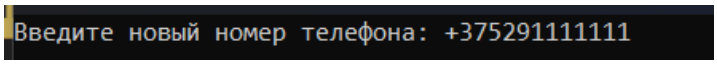


Рисунок 8.42 – Корректный ввод номера телефона

Также программа предусматривает ситуацию при смене пароля, новый пароль должен отличаться от старого (рисунок 8.43)



Рисунок 8.43 – Сообщение о некорректном вводе нового пароля

При вводе времени занятий предусмотрена проверка. Время конца занятия не должно быть раньше начала занятия (рисунок 8.44).

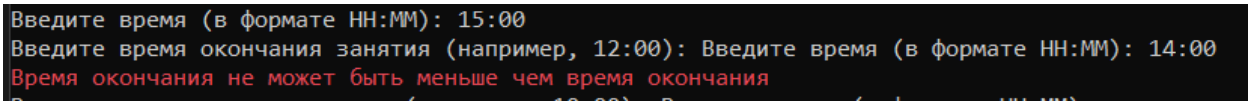


Рисунок 8.44 – Сообщение о некорректном вводе времени занятия

Также в программе реализован выход из меню по нажатию клавиши Esc, при этом, если пользователь нажмет любую другую клавишу, кроме Esc, приложение не произведет выход из меню.

Проведенное тестирование показало, что приложение успешно обрабатывает большинство возможных исключений и ошибок. Все основные сценарии использования, включая ошибочные действия со стороны пользователя, покрыты механизмами обработки ошибок. Программа не завершает свою работу с критическими сбоями, а вместо этого информирует пользователя о возникших проблемах и предлагает возможные пути их решения.

На основании тестирования можно сделать вывод, что автоматизированная система учета деятельности школы иностранных языков функционирует стабильно и устойчиво к ошибкам, выполнены все необходимые требования для надежной работы приложения.

Автоматизированная система учета деятельности школы иностранных языков является эффективной. Высокая эффективность заключается в оптимизации процессов управления, что позволяет существенно снизить нагрузку на администраторов и преподавателей. Внедрение такой системы способствует улучшению организации образовательного процесса и повышению качества обслуживания клиентов [8].

ЗАКЛЮЧЕНИЕ

В ходе выполнения данного курсового проекта были успешно решены все поставленные цели и задачи. На протяжении работы был проведен всесторонний анализ существующих информационных систем, выявлены проблемы и недостатки традиционных методов учета в образовательных учреждениях, что позволило обосновать необходимость разработки специализированного решения для школы иностранных языков.

Целью работы было создание эффективной автоматизированной системы, которая обеспечит учет всех аспектов деятельности учебного заведения, включая управление расписанием, взаимодействие с преподавателями и студентами и многое другое. В результате разработки была предложена система, отвечающая этим требованиям и интегрирующая различные рабочие процессы в единую платформу.

В процессе выполнения работы были решены следующие задачи.

Анализ существующих решений. Был проведен анализ существующих программных решений для автоматизации учебных заведений, что позволило выделить как положительные, так и отрицательные стороны текущих систем. Это помогло разработать систему, которая учитывает все потребности школы иностранных языков и исправляет недостатки существующих решений.

Проектирование архитектуры системы. На основании анализа была спроектирована архитектура системы, которая включает в себя несколько функциональных модулей. Каждый из этих модулей был спроектирован с учетом требований к удобству использования и быстроте обработки данных.

Выбор технологий для реализации системы. Для разработки системы были выбраны современные и надежные технологии, которые обеспечивают высокую производительность и безопасность.

Создание пользовательского интерфейса. Одной из ключевых задач было создание интуитивно понятного и удобного интерфейса для пользователей системы (администраторов, преподавателей, студентов). Все элементы интерфейса были продуманы таким образом, чтобы обеспечить удобство работы с системой и минимизировать вероятность ошибок при вводе данных.

Оценка эффективности системы. В заключение работы была проведена оценка эффективности внедрения предложенной системы. Ожидаемые результаты включают снижение временных затрат на выполнение рутинных задач, уменьшение количества ошибок в учете, повышение качества обслуживания студентов и преподавателей, а также улучшение общей организационной структуры школы.

Таким образом, все цели, поставленные в начале работы, были успешно достигнуты. В результате разработки была предложена комплексная автоматизированная система, которая значительно улучшит процессы учета и управления в школе иностранных языков. Внедрение данной системы позволит не только улучшить взаимодействие между студентами и преподавателями, но и оптимизировать работу административного персонала, сократив время, затрачиваемое на выполнение рутинных операций.

Разработанная система имеет высокую гибкость и может быть адаптирована под различные требования и особенности других образовательных учреждений. В дальнейшем возможна доработка и расширение функционала системы, например, для работы с более широким спектром образовательных курсов или для интеграции с другими информационными системами.

Результаты данного курсового проекта могут быть полезны не только для школы иностранных языков, но и для других образовательных учреждений, стремящихся повысить свою эффективность и снизить нагрузку на административный персонал.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Иностранные языки и развитие мозга человека [Электронный ресурс]. – Режим доступа: https://osvita.in/ru/inostrannye-jazyki-i-razvitie-mozga-cheloveka-porazitelnye-fakty/

[2] Система Moodle [Электронный ресурс]. – Режим доступа: https://mts-link.ru/blog/moodle/

[3] Что такое CRM Битрикс 24 [Электронный ресурс]. – Режим доступа: https://www.aviant.ru/about/articles/chto-takoe-crm-bitriks-24/

[4] 1С:Образование [Электронный ресурс]. – Режим доступа: https://obr.1c.ru/educational/prepodavatelyam/1s-obrazovanie-5-shkola/

[5] Азы моделирования в IDEF0 [Электронный ресурс]. – Режим доступа: https://www.comindware.ru/blog/азы-моделирования-в-idef0/

[6] Сущности и связи: как и для чего системные аналитики создают ER-диаграммы [Электронный ресурс]. – Режим доступа: https://practicum.yandex.ru/blog/chto-takoe-er-diagramma/

[7] Меженная, М. М. Основы конструирования программ: пособие / М. М. Меженная. – Минск : БГУИР, 2019. – 80 с.

[8] Шилдт Г. Самоучитель C++: Пер.с англ. – 3-е изд. / Г. Шилдт – СПб.: БХВ-Петербург, 2006. – 688 с.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программы с комментариями

```
#include <iostream>
#include <vector>
#include <locale>

using namespace std;

#include "UserValidation.h"

int main() {
     setlocale(LC_ALL, "Ru");
     system("chcp 1251 > nul");
     std::vector<Course> courses;
     std::vector<Student> students;
     std::vector<Teacher> teachers;
     std::vector<Admin> admins;
     std::vector<Certificate> certificates;
     FileService::readStudentsFromFile("students.txt",
students);
     FileService::readTeachersFromFile("teachers.txt",
teachers);
     FileService::readCoursesFromFile("courses.txt",
courses);
     FileService::readCertificatesFromFile("certificate.txt"
, certificates);

     FileService::readAdminsFromFile("admins.txt", admins);

     if (admins.empty()) {
          FileService::createAdminUser("admins.txt");
          FileService::readAdminsFromFile("admins.txt",
admins);
     }

     // Основной цикл программы
     while (true) {
          int choice;
          std::cout << "1. Войти" << std::endl << std::endl;
          std::cout << "2. Зарегистрироваться" << std::endl
<< std::endl;
```

```
                std::cout << "3. Выйти из системы" << std::endl <<
std::endl;
                std::cout << "Выберите действие: ";
                choice = ValidationManager::numberControl();

                 //Выполнение выбранного действия
                if (choice == 1) {
                     system("cls");
                     UserValidation::login(students,admins,
courses, teachers, certificates); // Вход пользователя
                }
                else if (choice == 2) {
                     system("cls");
                     UserValidation::registerUser(students,
teachers); // Регистрация нового пользователя
                }
                else if (choice == 3) {
                     system("cls");
                     break; // Выход из программы
                }
                else {
                     system("cls");

     SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),
LightBlue);
                     std::cout << "Некорректный ввод. Попробуйте
еще раз." << std::endl<<std::endl;

     SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),
LightGray);
                }
          }

          FileService::writeAdminsToFile(admins, "admins.txt");
          FileService::writeTeachersToFile("teachers.txt",
teachers);
          FileService::writeStudentsToFile("students.txt",
students);
          FileService::writeCoursesToFile("courses.txt",
courses);
```

```
        FileService::writeCertificatesToFile("certificate.txt",
certificates);

        return 0;
    }

    #ifndef ORDER_C
    #define ORDER_C
    #include "Course.h"
    #include <iostream>
    #include <iomanip>


    // Инициализация статической переменной для генерации ID
    std::unordered_set<unsigned int> Course::IDs = {};

    // Конструктор класса Course
    Course::Course(const std::string& name, const std::string&
level, int teacherID, int totalTopics, Language language, const
Schedule& schedule, int totalStudent)
        : courseID(generateID()), name(name), level(level),
teacherID(teacherID),language(language),
        totalTopics(totalTopics), completedTopics(0),
isCompleted(false), totalStudents(totalStudent),
schedule(schedule) {
    }

    Course::Course(const std::string& name, const std::string&
level, int teacherID, int totalTopics,
        int completedTopics, bool isCompleted, int
totalStudents,
        const std::vector<int>& studentsEnrolledID, Language
language, const Schedule& schedule)
        : courseID(generateID()), name(name), level(level),
teacherID(teacherID),
        totalTopics(totalTopics),
completedTopics(completedTopics), isCompleted(isCompleted),
        totalStudents(totalStudents),
studentsEnrolledID(studentsEnrolledID),
        language(language), schedule(schedule) {
    }
```

```
     Course::Course(const int& courseID,const std::string& name,
const std::string& level, int teacherID, int totalTopics,
          int    completedTopics,    bool    isCompleted,    int
totalStudents,
          const  std::vector<int>&  studentsEnrolledID,  Language
language, const Schedule& schedule)
          :    courseID(courseID),    name(name),    level(level),
teacherID(teacherID),
          totalTopics(totalTopics),
completedTopics(completedTopics), isCompleted(isCompleted),
          totalStudents(totalStudents),
studentsEnrolledID(studentsEnrolledID),
          language(language), schedule(schedule) {
          IDs.insert(courseID);
     }
     // Геттеры и сеттеры

     // Получить ID курса
     int Course::getCourseID() const {
          return courseID;
     }

     // Установить ID курса
     void Course::setCourseID(int id) {
          courseID = id;
     }
     // Получить название курса
     std::string Course::getName() const {
          return name;
     }

     // Установить название курса
     void Course::setName(const std::string& name) {
          this->name = name;
     }

     // Получить язык курса
     Language Course::getLanguage() const {
          return language;
     }

     // Установить язык курса
     void Course::setLanguage(Language language) {
```

```
     this->language = language;
}

// Получить уровень курса
std::string Course::getLevel() const {
     return level;
}

// Установить уровень курса
void Course::setLevel(const std::string& level) {
     this->level = level;
}

// Получить ID преподавателя курса
int Course::getID() const {
     return teacherID;
}

// Установить ID преподавателя курса
void Course::setTeacherID(int teacherID) {
     this->teacherID = teacherID;
}

// Получить общее количество тем на курсе
int Course::getTotalTopics() const {
     return totalTopics;
}

// Установить общее количество тем
void Course::setTotalTopics(int totalTopics) {
     this->totalTopics = totalTopics;

}

// Получить количество завершенных тем
int Course::getCompletedTopics() const {
     return completedTopics;
}

// Установить количество завершенных тем
void Course::setCompletedTopics(int completedTopics) {
     this->completedTopics = completedTopics;
```

```
}

// Проверить, завершен ли курс
bool Course::getIsCompleted() const {
    return isCompleted;
}

// Установить статус завершенности курса
void Course::setIsCompleted(bool isCompleted) {
    this->isCompleted = isCompleted;
}

// Получить количество студентов на курсе
int Course::getTotalStudents() const {
    return totalStudents;
}

// Установить количество студентов
void Course::setTotalStudents(int totalStudents) {
    this->totalStudents = totalStudents;
}

// Получить список ID студентов, записанных на курс
std::vector<int> Course::getStudentsEnrolledID() const {
    return studentsEnrolledID;
}

std::vector<int>& Course::getStudentsEnrolledID() {
    return studentsEnrolledID;
}

// Добавить студента на курс
void Course::addStudent(int studentID) {
    studentsEnrolledID.push_back(studentID);
    totalStudents++;
}

const Schedule& Course::getSchedule() const { return
schedule; }
void Course::setSchedule(const Schedule& schedule) {
    this->schedule = schedule;
}
```

```
     // Удалить студента с курса
     void Course::removeStudent(int studentID) {
          auto    it    =    std::find(studentsEnrolledID.begin(),
studentsEnrolledID.end(), studentID);
          if (it != studentsEnrolledID.end()) {
               studentsEnrolledID.erase(it);
               totalStudents--;
          }
     }

     // Метод для генерации уникального ID
     int Course::generateID() {
          int newID = 1000;
          do {
               ++newID;   // Генерируем новый ID
          } while (IDs.find(newID) != IDs.end());
          IDs.insert(newID);
          return newID;
     }

     std::string Course::languageToString(Language language) {
          switch (language) {
          case Language::English: return "Английский";
          case Language::Spanish: return "Испанский";
          case Language::French:  return "Французский";
          case Language::German:  return "Немецкий";
          case Language::Italian: return "Итальянский";
          case Language::Chinese: return "Китайский";
          case Language::Japanese: return "Японский";
          case Language::Russian: return "Русский";
          case Language::Arabic: return "Арабский";
          case Language::Portuguese: return "Португальский";
          default: return "Неизвестный язык";
          }
     }
     void Course::incrementCompletedTopics() {
          if (completedTopics < totalTopics) {
               ++completedTopics;
          }
     }

     void Course::markAsCompleted() {
          isCompleted = true;
```

```
}

Schedule::Schedule() {
}

std::vector<DayOfWeek> Schedule::getDay() const {
   return days;
}

std::string Schedule::getStartTime() const {
     return start_time;
}

std::string Schedule::getEndTime() const {
     return end_time;
}

std::string Schedule::getRoom() const {
     return room;
}

void Schedule::displaySchedule() const {
}

std::string Schedule::getDayAsString(DayOfWeek day) {
     switch (day) {
     case DayOfWeek::Monday: return "Понедельник";
     case DayOfWeek::Tuesday: return "Вторник";
     case DayOfWeek::Wednesday: return "Среда";
     case DayOfWeek::Thursday: return "Четверг";
     case DayOfWeek::Friday: return "Пятница";
     case DayOfWeek::Saturday: return "Суббота";
     case DayOfWeek::Sunday: return "Воскресенье";
     default: return "Неизвестный день";
     }
}

#endif

#include "Certificate.h"
#include <iostream>

// Конструктор
```

```
    Certificate::Certificate(const int courseID,const
std::string& teacherName, const std::string& courseName,
        Language language, const std::string& languageLevel)
        : courseID(courseID), teacherName(teacherName),
courseName(courseName), language(language),
languageLevel(languageLevel) {
    }

    int Certificate::getCourseID() const {
        return courseID;
    }

    // Геттеры
    std::string Certificate::getTeacherName() const {
        return teacherName;
    }

    std::string Certificate::getCourseName() const {
        return courseName;
    }

    Language Certificate::getLanguage() const {
        return language;
    }

    std::string Certificate::getLanguageLevel() const {
        return languageLevel;
    }

    void Certificate::setCourceID(const int id) {
        courseID = id;
    }

    // Сеттеры
    void Certificate::setTeacherName(const std::string&
teacherName) {
        this->teacherName = teacherName;
    }

    void Certificate::setCourseName(const std::string&
courseName) {
        this->courseName = courseName;
    }
```

```
    void Certificate::setLanguage(Language language) {
        this->language = language;
    }

    void    Certificate::setLanguageLevel(const    std::string&
languageLevel) {
        this->languageLevel = languageLevel;
    }

    // Метод для преобразования Language в строку
    std::string Certificate::languageToString(Language language)
{
        switch (language) {
        case Language::English: return "Английский";
        case Language::Spanish: return "Испанский";
        case Language::French:  return "Французский";
        case Language::German:  return "Немецкий";
        case Language::Italian: return "Итальянский";
        case Language::Chinese: return "Китайский";
        case Language::Japanese: return "Японский";
        case Language::Russian: return "Русский";
        case Language::Arabic: return "Арабский";
        case Language::Portuguese: return "Португальский";
        default: return "Неизвестный язык";
        }
    }

    // Метод для генерации текста сертификата
    std::string Certificate::generateCertificateText() const {
        return  "Сертификат  подтверждает  успешное  прохождение
курса \"" + courseName +
            "\" на уровне " + languageLevel + " по языку " +
languageToString(language) +
            ". Курс преподавал(а): " + teacherName + ".";
    }

    // Метод для отображения информации о сертификате
    void Certificate::displayCertificate() const {
        std::cout << "=== Сертификат ===" << std::endl;
        std::cout << generateCertificateText() << std::endl;
    }

    #pragma once
```

```
#include "User.h"
#include "Certificate.h"

enum RequestStatus {
     Rejected = -1,
     Expected,
     Approved
};

class Request;

class Student:public Account {
private:
     int studentID;
     std::vector<int> coursesID;
     std::string phoneNumber;
     std::string mail;
     std::vector<Request> requests;
     std::vector<Certificate> studentCertificate;

     static std::unordered_set<unsigned int> IDs;

public:
     Student();
     Student(const std::string& uname, const std::string&
pass, const std::string& slt, Acess& acesses, std::vector<int>
coursesID, const std::string& phoneNumber,const std::string&
mail);
     Student(const std::string& uname, const std::string&
hashedPass, const std::string& salt, Acess ac, int id, const
std::vector<int>& courses, const std::string& phone, const
std::string& mail, const std::vector<Request>& reqs);
     // Геттеры
     std::vector<int> getCoursesID() const;
     std::vector<int>& getCoursesID();
     int getID()const;
     std::string getPhoneNumber() const;
     std::string getMail() const;
     const std::vector<Request>& getRequests() const;
     std::vector<Request>& getRequests();
     // Сеттеры
     void setCoursesID(const std::vector<int>& coursesID);
     void setPhoneNumber(std::string& phoneNumber);
```

```
        void setMail(std::string& mail);

        bool hasCourse(int courseID) const;

        int generateID();

        const                          std::vector<Certificate>&
getStudentCertificate()const;

        std::vector<Certificate>& getStudentCertificate();

        void removeCourse(int courseID);


        void   openStudentMenu(std::vector<Course>&   courses,
std::vector<Certificate>&          certificates,          const
std::vector<Student>& students);
        void changePassword();
        void   displayCourseTable(const   std::vector<Course>&
courses) const; // Таблица с форматированием
        void  sortCourses(std::vector<Course>&  courses,  int
category, bool ascending); // Сортировка курсов
        void displayForFind(const std::vector<Course>& courses,
std::string& str);
        void          viewAvailableCourses(std::vector<Course>&
courses); // Просмотр курсов с сортировкой

        void findCourse(const std::vector<Course>& courses);

        void   enrollInCourse(std::vector<Request>&   requests,
const std::vector<Course>& courses);
        void viewMyCourses(const std::vector<Course>& courses)
const;
        void  viewSchedule(const  std::vector<Course>&  courses)
const;
        void  viewRequests(const  std::vector<Course>&  courses)
const;
        void viewCertificates() const;
        void editProfile();

    };
    class Request {
    private:
```

```
        int requestId;
        int courseID;
        RequestStatus status;

        static int nextID;

    public:

        Request(int reqId, const int courseId, RequestStatus
stat);
        Request(const int courseId, RequestStatus stat);

        RequestStatus getStatus() const;
        int getRequestID() const;
        int getCourseID() const;

        void setStatus(RequestStatus newStatus);

        static    std::string    statusToString(RequestStatus
status);
```

ПРИЛОЖЕНИЕ Б
(обязательное)
Диаграмма классов

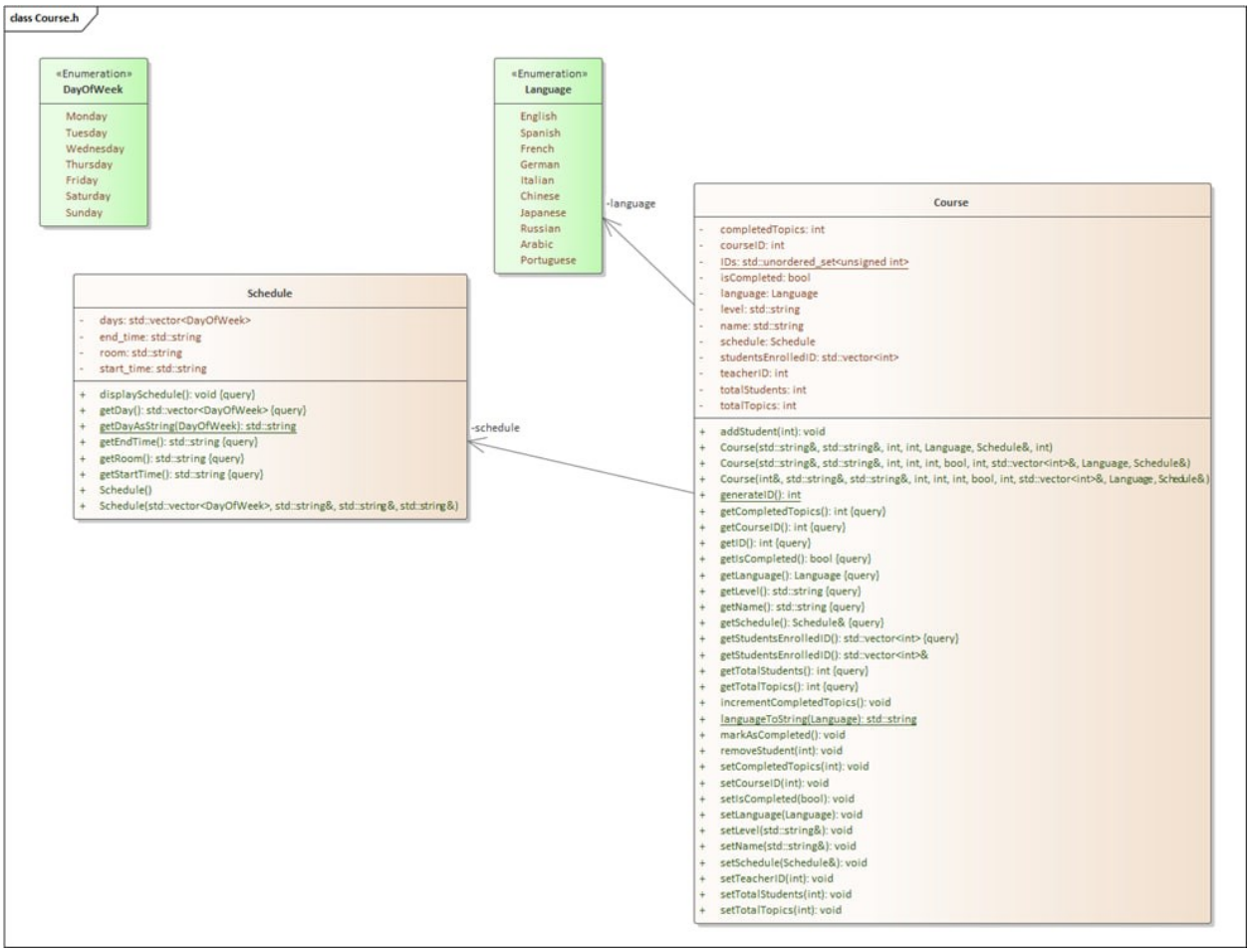


Рисунок Б.1 – Диаграмма классов, часть 1



Рисунок Б.2 – Диаграмма классов, часть 2



Рисунок Б.3 – Диаграмма классов, часть 3



Рисунок Б.4 – Диаграмма классов, часть 4

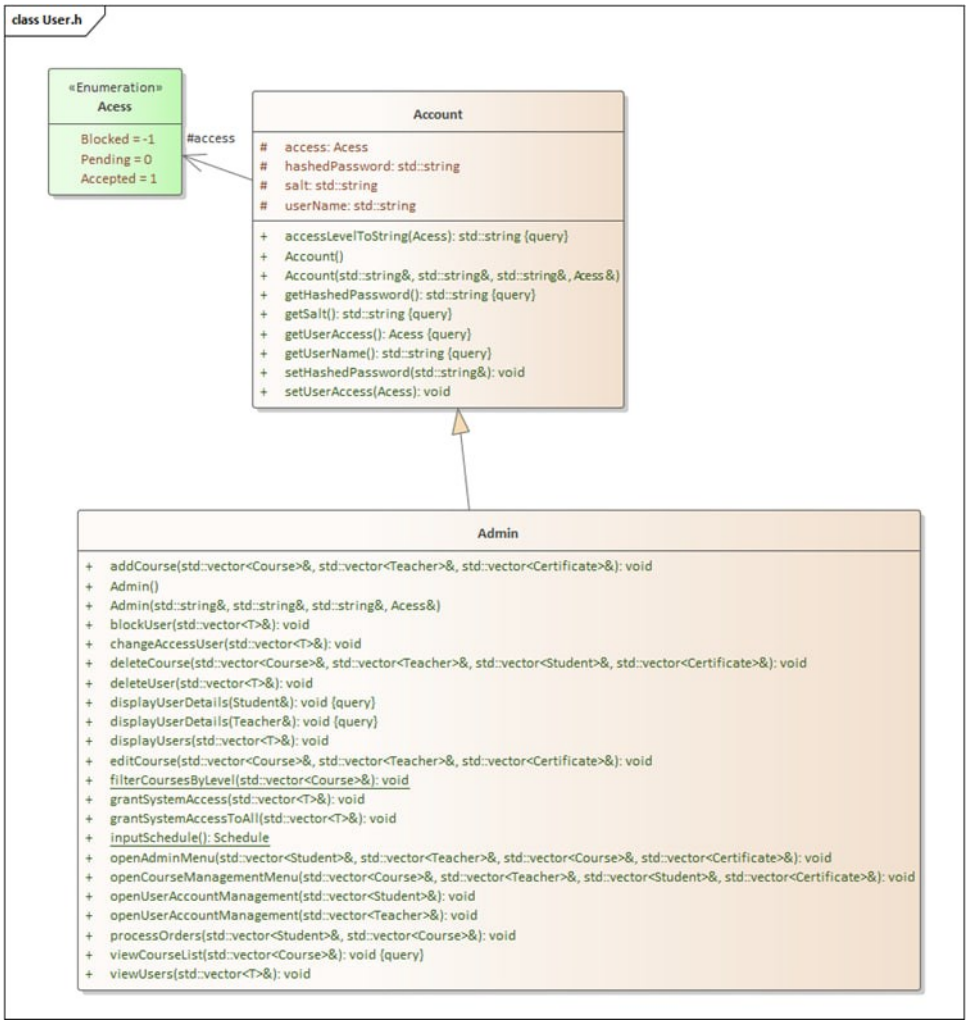


Рисунок Б.5 – Диаграмма классов, часть 5

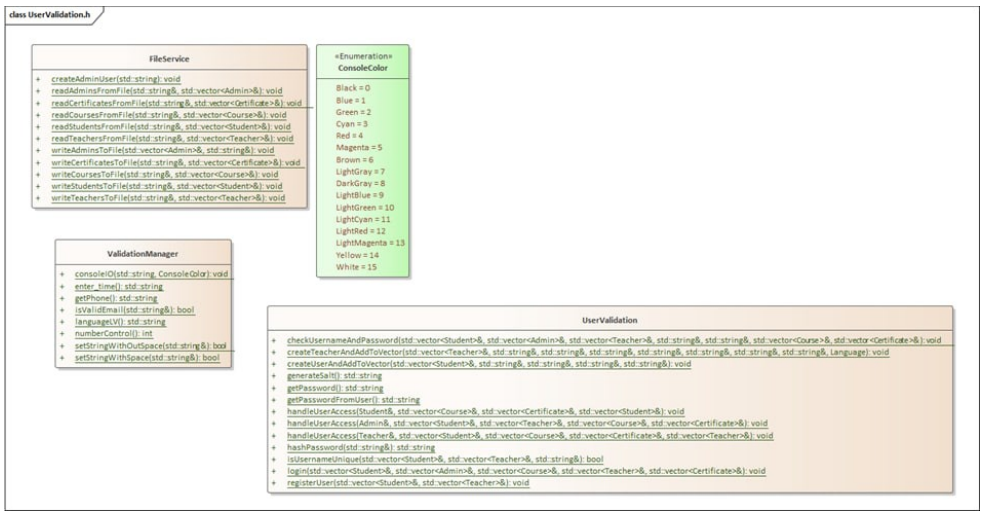


Рисунок Б.6 – Диаграмма классов, часть 6

ПРИЛОЖЕНИЕ В
(обязательное)
Диаграмма вариантов использования

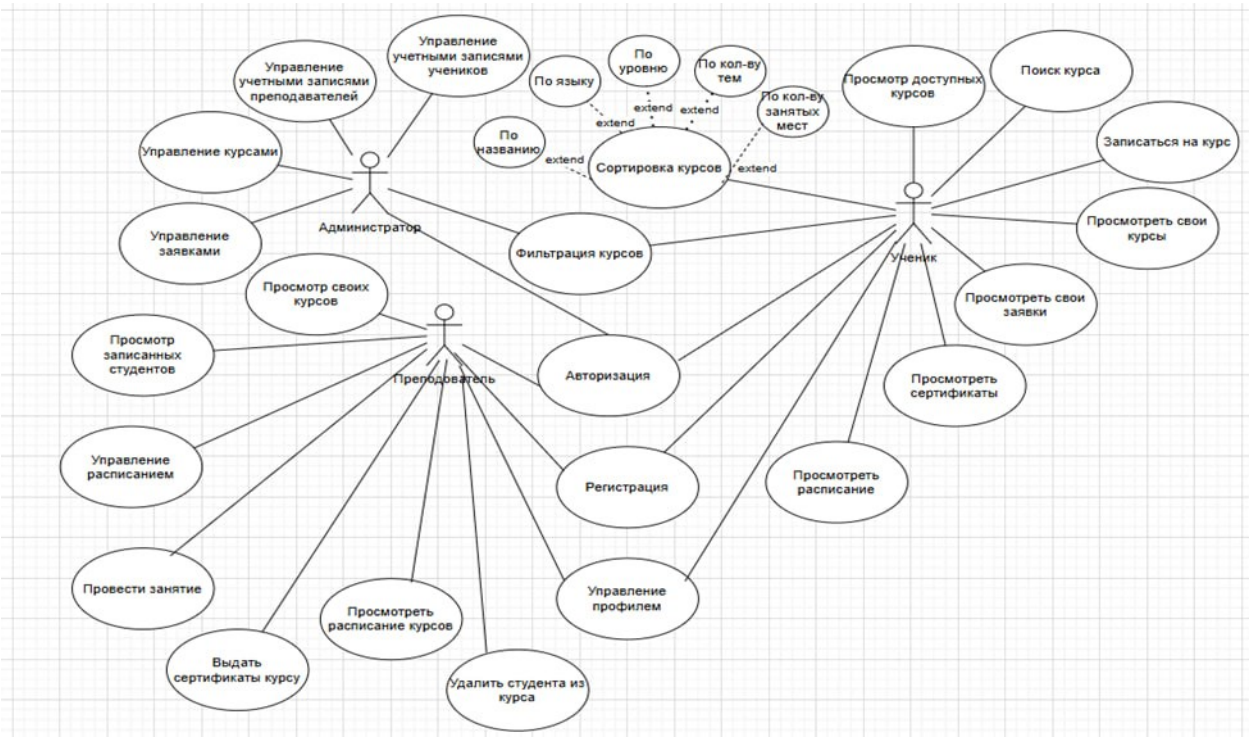


Рисунок В.1 – Диаграмма вариантов использования приложения

ПРИЛОЖЕНИЕ Г
(обязательное)
Блок-схемы алгоритмов

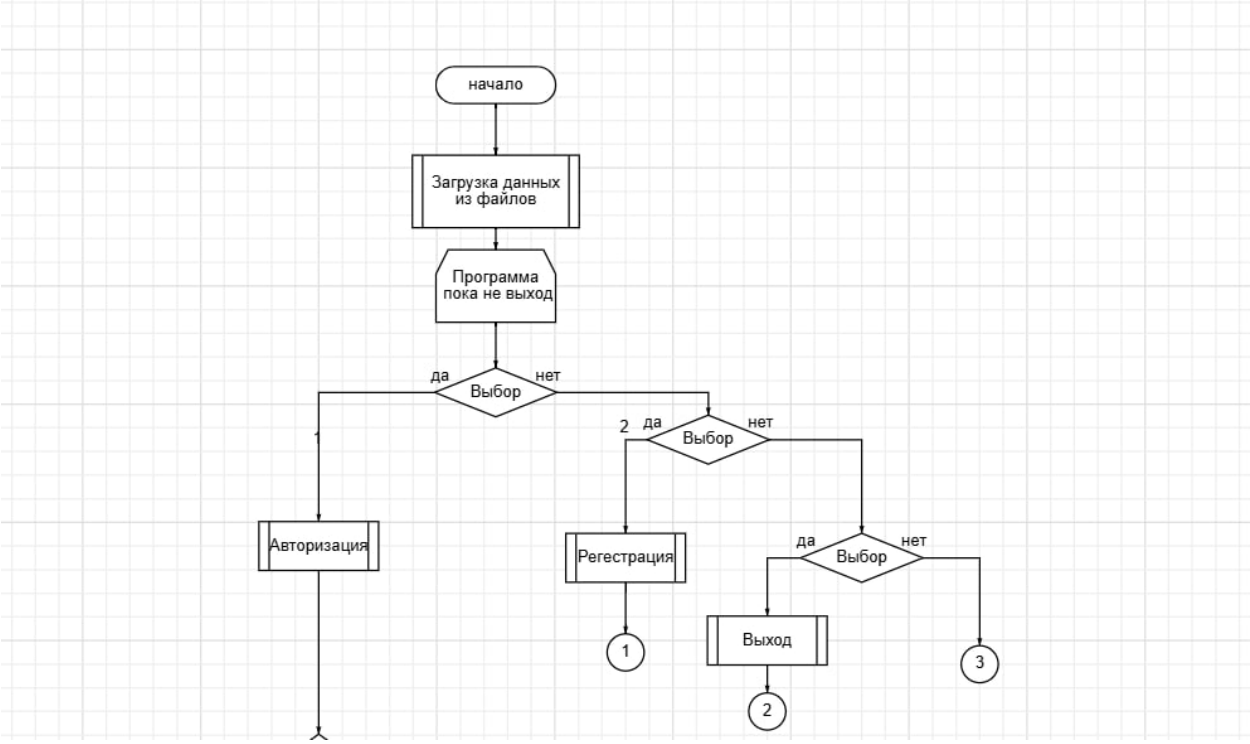


Рисунок Г.1 – Блок-схема алгоритма работы всей программы

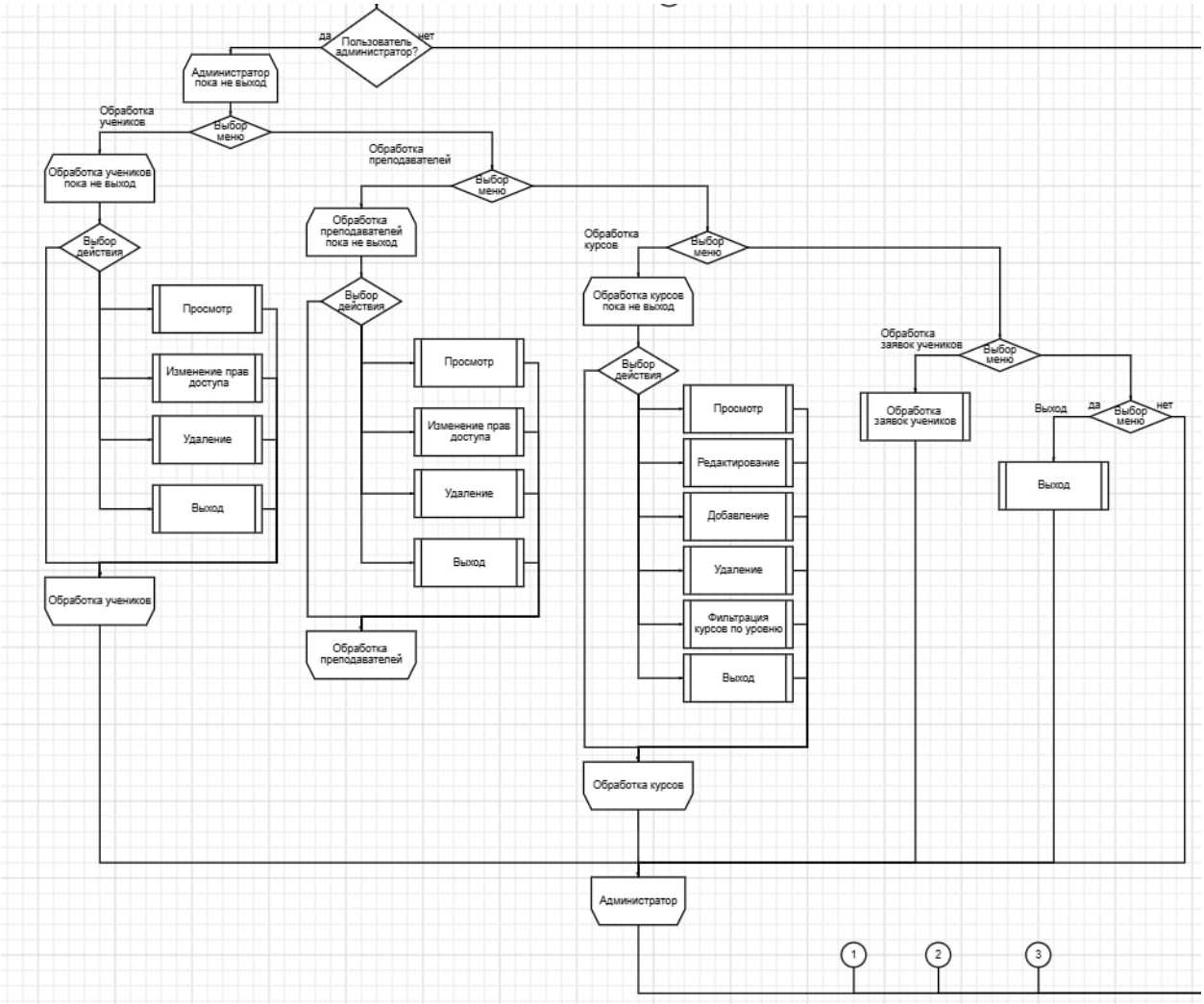


Рисунок Г.2 - Продолжение схемы алгоритма работы всей программы

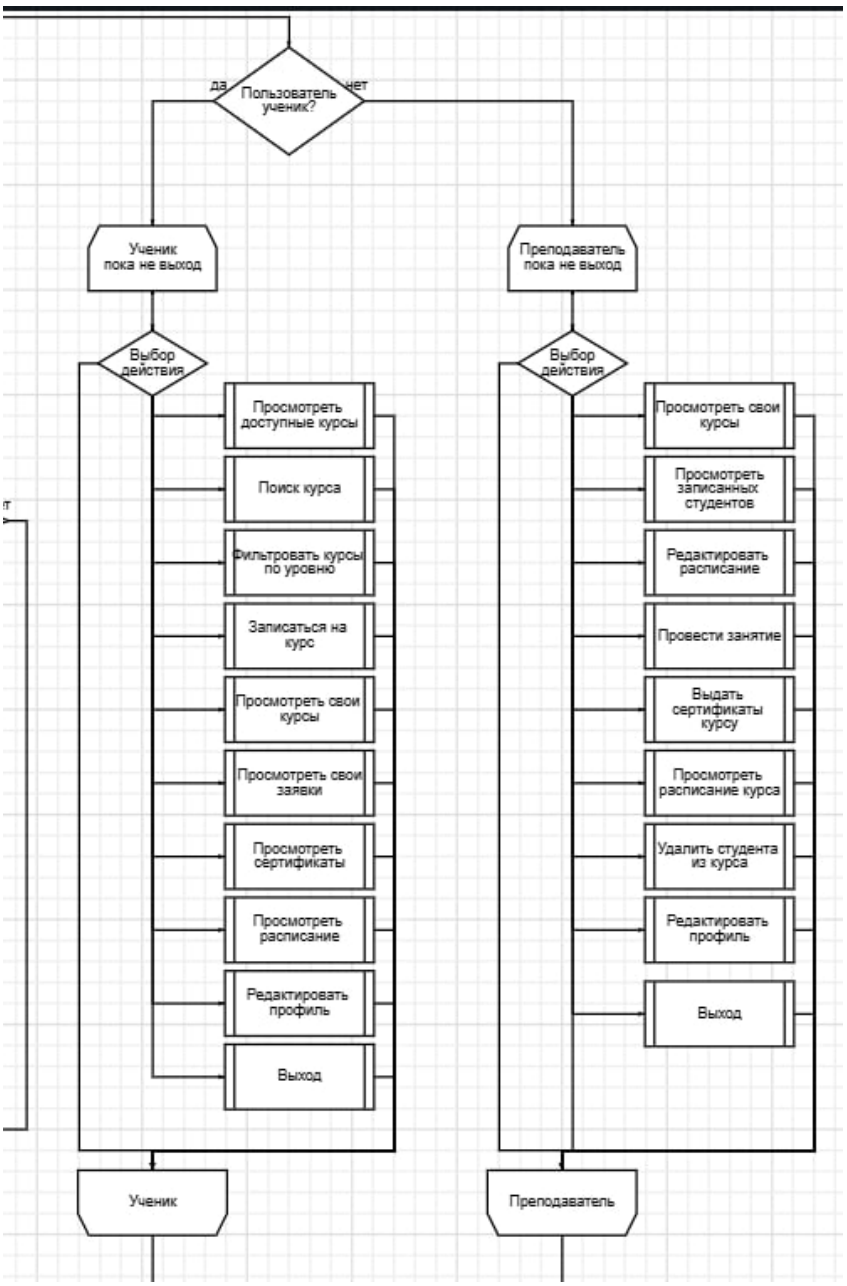


Рисунок Г.3 - Продолжение схемы алгоритма работы всей программы



Рисунок Г.4 – Продолжение схемы алгоритма работы всей программы

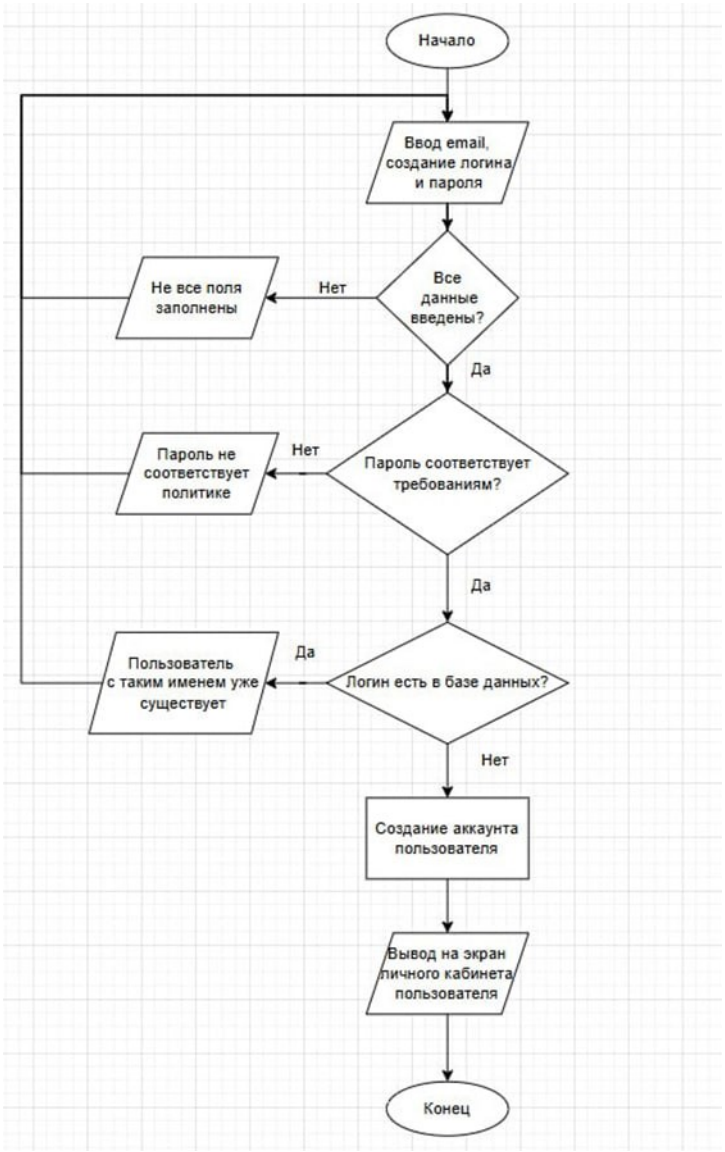


Рисунок Г.5 – Схема алгоритма регистрации пользователя

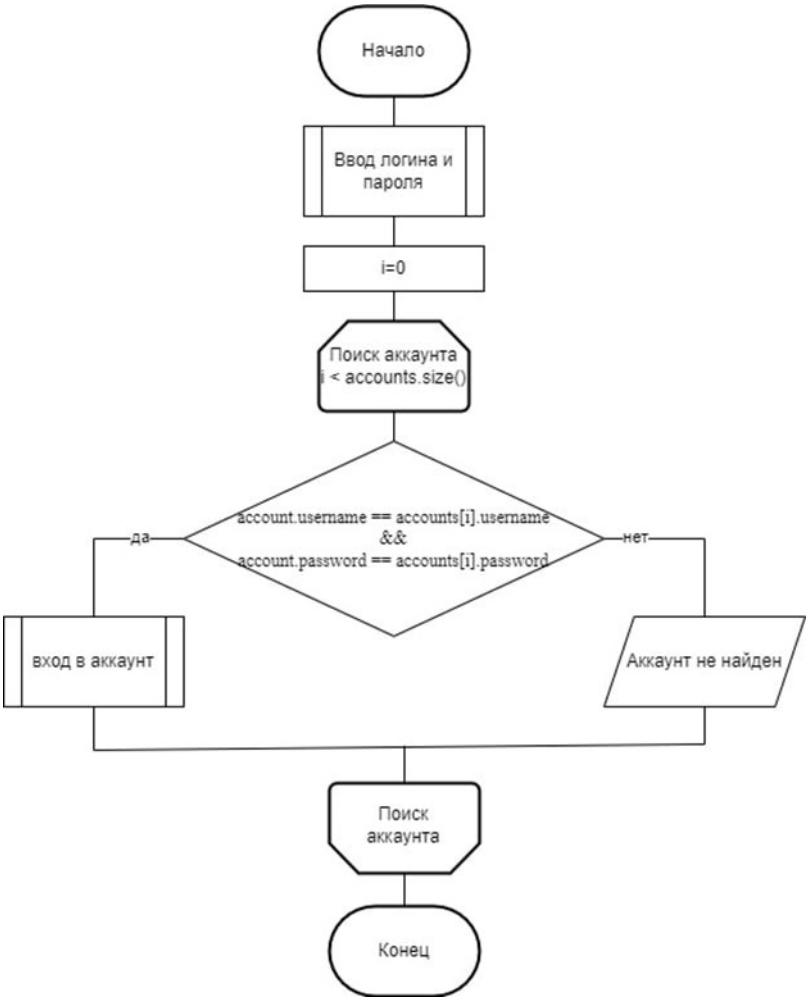


Рисунок Г.6 – Схема алгоритма авторизации пользователя